FAKULTA MECHATRONIKY,
INFORMATIKY A MEZIOBOROVÝCH
STUDIÍ TUL

Diplomová práce

Architektura systému pohledové inspekce kvality

Studijní program: N0613A140028 – Informační technologie
Studijní obor: N0613A140028AI – Aplikovaná informatika
Autor práce: **Bc. Kevin Daněk**
Vedoucí práce: Ing. Igor Kopetschke

Liberec 2025

Tento list nahraďte
originálem zadání.

PROHLÁŠENÍ

Prohlašuji, že svou diplomovou práci jsem vypracoval samostatně jako původní dílo s použitím uvedené literatury a na základě konzultací s vedoucím mé diplomové práce a konzultantem.
Jsem si vědom toho, že na mou diplomovou práci se plně vztahuje zákon č. 121/2000 Sb., o právu autorském, zejména § 60 – školní dílo.
Beru na vědomí, že Technická univerzita v Liberci nezasahuje do mých autorských práv užitím mé diplomové práce pro vnitřní potřebu Technické univerzity v Liberci.
Užiji-li diplomovou práci nebo poskytnu-li licenci k jejímu využití, jsem si vědom povinnosti informovat o této skutečnosti Technickou univerzitu v Liberci; v tomto případě má Technická univerzita v Liberci právo ode mne požadovat úhradu nákladů, které vynaložila na vytvoření díla, až do jejich skutečné výše.
Současně čestně prohlašuji, že text elektronické podoby práce vložený do IS STAG se shoduje s textem tištěné podoby práce.
Beru na vědomí, že má diplomová práce bude zveřejněna Technickou univerzitou v Liberci v souladu s § 47b zákona č. 111/1998 Sb., o vysokých školách a o změně a doplnění dalších zákonů (zákon o vysokých školách), ve znění pozdějších předpisů.
Jsem si vědom následků, které podle zákona o vysokých školách mohou vyplývat z porušení tohoto prohlášení.

3. 5. 2026 Bc. Kevin Daněk

ARCHITEKTURA SYSTÉMU POHLEDOVÉ INSPEKCE KVALITY

ABSTRAKT

Tato zpráva popisuje třídu `tulthesis` pro sazbu absolventských prací Technické univerzity v Liberci pomocí typografického systému LaTeX.

Klíčová slova: LaTeX, třída, TUL

ABSTRACT

This report describes the `tulthesis` package for Technical university of Liberec thesis typesetting using the LaTeX typographic system.

Keywords: LaTeX, class, TUL

PODĚKOVÁNÍ

V první řadě bych chtěl poděkovat vedoucímu mé práce, panu Ing. Igorovi Kopetschkovi, který mé bláznivé kvalifikační práce zastřešuje po celou dobu mého studia. Jeho lidský a pragmatický přístup mi dával naději, že nemusím hrát v absurdnímu dramatu akademické aristokracie - a že je nejenom možné, ba dokononce žádoucí, umět podat odborné téma lidsky a srozumitelně. Mrzí mě, že kdybych se rozhodl pokračovat do vyšších pater akademického studia, nebude mě moci doprovázet. Možná je to ale signálem, že je čas uvolnit místo pro ostatní, a věřte, že pokud vložíte v pana Koptschkeho důvěru, rozhodně nebudete strádat.
Dál bych chtěl poděkovat i zbytku akademické obce, se kterou jsem měl co dočinění. I když se s některými jejími členy, ozvláště pak z nadřízených pozic, neshodnu ani na počasí, tak díky každému jsem získal novou zkušenost. Někdy byla pozitivní, a někdy zase i negativní - to ale k životu patří.
A na závěr musím poděkovat svojí drahé polovičce, Simče Daňkové, která mě podporovala po celou dobu psaní této práce. Na začátku této práce byla mou přítelkyní, a nyní, na jejím konci, je již mojí manželkou. Díky ní si budu tuto práci pamatovat nejenom jako svůj *magnum opus*, ale jako životní milník, kterým se změnil skoro každý aspekt mého života. Pokud Vás zajímají regionální dějiny a příběhy z časů minulých, doporučuji si přečíst její bakalářskou práci, kterou tvoří souběžně s tou mojí, a zabývá se tématem sebevražd po druhé světové válce na Liberecku a obzvláště v Jablonci nad Nisou a jeho blízkém okolí.

OBSAH

Obsah 6
Seznam zkratek 10

1 Úvod **11**

2 Softwarové inženýrství **12**
2.1 Technický dluh 13
2.1.1 Vznik technického dluhu 14
2.1.2 Nešváry 14
2.1.3 Detekce technického dluhu 16
2.1.4 Řešení technického dluhu 17
2.2 Clean Code 18
2.2.1 Význam a záměr identifikátorů 18
2.2.2 Pravdivost a jednoznačnost identifikátorů 19
2.2.3 Sémantické odlišení identifikátorů 19
2.2.4 Hledatelnost a verbální komunikace 20
2.2.5 Očištění od technického šumu 20
2.2.6 Vlastnosti funkcí a metod 20
2.2.7 Délka a hloubka zanoření 20
2.2.8 Jednotná úroveň abstrakce 21
2.2.9 Pravidlo sestupného čtení 21
2.2.10 Rozhraní a počet argumentů 21
2.2.11 Seskupování argumentů a příznaky 22
2.3 Návrhové vzory 22
2.3.1 Specifikace návrhových vzorů 22
2.3.2 Specifikace pomocí teorie kategorií 25
2.3.3 Popis vybraných návrhových vzorů 29
2.3.4 Kompozitní vzory 37
2.3.5 Závěr 39
2.4 Architektonické vzory 40
2.4.1 Koncept softwarové architektury 40
2.4.2 Popis vybraných architektonických vzorů 41
2.4.3 Závěr 45
2.5 Integrační vzory 45
2.5.1 Popis vybraných integračních vzorů 46
2.6 Závěry 47

3 Udržitelnost produkčního kódu **48**
3.1 Testování 48
3.2 Observabilita 50
3.2.1 Logy 50
3.2.2 Metriky 51
3.2.3 Trasy 51

4 Audit architektury systému **52**
4.1 Modelování softwarové architektury 53
4.1.1 Úroveň systémového kontextu 53
4.1.2 Úroveň kontejnerů 54
4.1.3 Úroveň komponent 55
4.2 Vyhodnocení rizik 56
4.2.1 Závažnost vady 56
4.2.2 Výskyt vady 57
4.2.3 Detekce vady 57
4.2.4 Zpracování analýzy 59
4.3 Modelování doménových procesů 59

5 Architektura nového řešení **60**
5.1 Postupné nasazení 60

6 Závěr **61**

Dodatek A Test **65**

SEZNAM OBRÁZKŮ

2.1 Rychlost vývoje v závislosti na složitosti návrhu 13
2.2 Kvadranty technického dluhu (Team Asana, 2025) 14
2.3 Kvadranty řešení technického dluhu (ForrestKnight, 2025) . . 17
2.4 Nepravdivé názvy vytvářejí překážky ve vývoji 19
2.5 Semantické identifikátory implikující vazby mezi objekty 19
2.6 Komutativní diagram 28
2.7 Funktor 28
2.8 Komutativní diagram návrhového vzoru: Factory 30
2.9 Komutativní diagram návrhového vzoru: Builder 31
2.10 Komutativní diagram návrhového vzoru: Singleton 32
2.11 Komutativní diagram návrhového vzoru: Facade 33
2.12 Komutativní diagram návrhového vzoru: COR 34
2.13 Komutativní diagram návrhového vzoru: Strategy 35
2.14 Komutativní diagram návrhového vzoru: Observer 36
2.15 Kompozitní návrhový vzor: Factory of Strategies 38
2.16 Kompozitní návrhový vzor: Factory of Strategies 39
2.17 Komutativní diagram vrstvené architektury 41
2.18 Komutativní diagram hexagonální architektury 43
2.19 Komutativní diagram událostmi řízené architektury 44

3.1 Úrovně abstrakce a typy testů 49

4.1 Diagram systémového kontextu 53
4.2 Diagram komponent 54
4.3 Diagram komponent kontejneru CCM UNI 55

SEZNAM TABULEK

2.1 Výběr nešvarů, jejich symptonů a příčin 16
2.2 Propojení úrovní modelu C4 s principy softwarového inženýrství 47

4.1 Adaptovaná hodnotící škála závažnosti vady 57
4.2 Adaptovaná hodnotící škála výskytu vady 58
4.3 Adaptovaná hodnotící škála detekce vady 58

SEZNAM ZKRATEK

CD Continuous Deployment (Postupné nasazení)

CI Continuous Integration (Postupná integrace)

MES Manufacturing Execution System

IDE Integrated Development Environment (Integrované vývojové prostředí)

1 ÚVOD

Každý, jakkoliv velký, systém musí někdo navrhnout. S rostoucí složitostí systému zároveň roste i složitost jeho správného návrhu. Ve své předchozí práci jsem se věnoval rapidnímu vývoji algoritmů strojového vidění, jehož výsledkem byla nová funkční komponenta pro trénování a inferenci neuronových sítí a jiných algoritmů strojového vidění. (Daněk, 2024)

Již tehdy byla samotná implementace omezena návrhem existujícího systému. Upřednostňování rychlého dodání nových funkcionalit na úkor průběžné údržby kódu vyústilo v enormní množství technologického dluhu, který se za sedm let ve zdrojovém kódu nastřádal. Tato práce navazuje na mou předchozí práci a dívá se na ní ze širší perspektivy. Cílem je odklonit se od vývoje nových funkcí a obrátit pozornost právě k zanedbané architektuře celého řešení. Rozsah analýzy jsem stanovil nad rámec softwarového návrhu a věnuji se rovněž architektuře infrastruktury nasazeného řešení.

2 SOFTWAROVÉ INŽENÝRSTVÍ

Programátor vybavený klávesnicí má téměř neomezené možnosti realizace svých nápadů, podobně jako malíř se štětcem. Je však pozoruhodné, že na rozdíl od nekontrolovaného tvůrčího procesu malíři často své možnosti úmyslně limitují. Právě dodržením určitých kompozičních pravidel totiž docílí toho, že obraz není pouze chaotickou směsí čar a barev, ale srozumitelnou myšlenkou, kterou divák dokáže instinktivně rozpoznat.

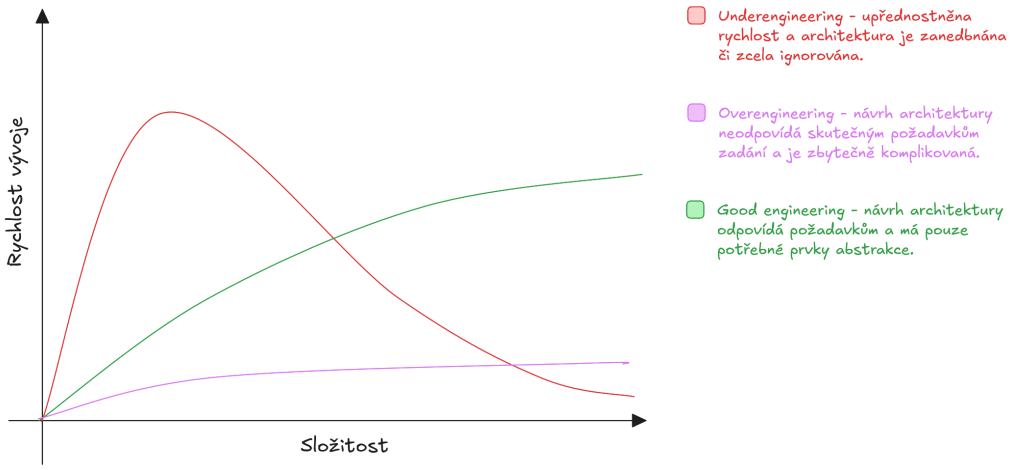
Principy vývoje softwaru se od této úvahy příliš neliší. Pokud se při vývoji zavážeme k dodržování stanovených pravidel a konvencí, umožníme ostatním vývojářům snadněji pochopit záměr našeho kódu. Této oblasti se právě věnuje **softwarové inženýrství**, které se zabývá návrhem kódu, který je udržitelný, čitelný, spravovatelný a testovatelný (Michigan Technological University, 2025). Z předvídatelného a čitelného kódu profituje i samotný autor, který se tak při návratu k vlastní práci po delším časovém odstupu nemusí věnovat náročnému dekódování původního záměru.

Softwarové inženýrství je velmi rozsáhlý obor a není v možnostech této práce jej pojmout v jeho úplnosti. Teoretický rámec se proto omezuje na vybraná témata, která jsou klíčová pro pochopení praktické části. Kapitolu otevírá pojednání o technickém dluhu, jeho příčinách a metodách jeho eliminace. Na tento koncept navazuje pasáž věnovaná principům udržitelného a testovatelného kódu, které tvoří nezbytný základ pro následné návrhové vzory. Ty využívají zmíněné principy a transformují je do osvědčených řešení běžných architektonických problémů.

Další část práce se věnuje metodice efektivního testování a principům kontinuální integrace a doručování (CI/CD). Teoretický blok uzavírá kapitola o monitoringu nasazeného řešení, který poskytuje nezbytný vhled do chování systému v produkčním prostředí, kde vývojář zpravidla nemá k dispozici stejné diagnostické nástroje jako v lokálním vývojovém prostředí.

2.1 TECHNICKÝ DLUH

Při vývoji software musí být učiněna určitá rozhodnutí o jeho návrhu. Ta jsou ovlivněna mnoha faktory - od hloubky znalosti domény a specifik zadání až po technologická omezení a časový tlak. Výsledná architektura je pak nevyhnutelně kompromisem mezi okamžitou rychlostí dodání a dlouhodobou udržitelností systému. Pod tlakem termínů musí vývojář neustále vyvažovat míru investice do čitelnosti, abstrakce a budoucí rozšířitelnosti kódu na úkor času, který má aktuálně k dispozici.



Obrázek 2.1: Rychlost vývoje v závislosti na složitosti návrhu

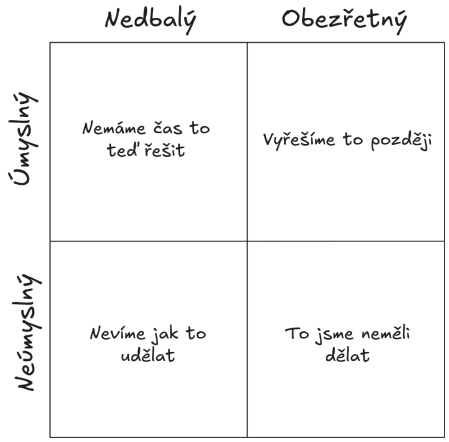
Pokud je zvolena cesta rychlejšího dodání namísto systematického návrhu, vzniká kód, který je tvořen ad hoc a postrádá ucelenou vizi. Absence celkového nadhledu pak způsobuje, že se systém s každou další funkcí stává nepřehlednějším a jeho údržba vyžaduje stále větší úsilí. Tento jev nazývám technickým dluhem. Pokud se technický dluh nechá střádat, postupně transformuje triviální změny na netriviální.

Trivialní změnou myslím takovou změnu, kdy není potřeba zasahovat do řídících struktur a logiky existujícího systému. Naopak netriviální změny vyžadují hlubší pozorumnění a studium existujícího kódu a jsou tím pádem nejen časově náročnější, ale i více zatíženy možnostmi pro vznik chyby. Příkladem triviální změny je přidání záznamu do slovníku dostupných algoritmů, nebo přidání nové obslužné metody pro údalost. Naopak příklad netriviální změny je aktualizace výrazů *switch* nebo *if* pro přidání nového algoritmu. V tento okamžik je totiž potřeba hlídat, že jsou tyto řídící struktury aktualizovány na všech relevantních místech.

Technický dluh se nazývá dluhem právě proto, že vykazuje stejné charakteristiky jako finanční závazek. Výměnou za rychlejší uvedení produktu na trh si vývojový tým od systému obrazně vypůjčuje čas, a to na úkor technické preciznosti. Stejně jako u finančních závazků, i zde je nutné počítat s úroky v podobě degradace kvality kódu.

2.1.1 Vznik technického dluhu

Vznik technického dluhu popisuje ve svých knihách a článcích Martin Fowler, který rozděluje příčiny vzniku technického dluhu do čtyř kategorií. Tyto kategorie leží na kvadrantu, který je založen na dvou osách: ose obezřetnosti a ose úmyslnosti. (Fowler, 2009)



Obrázek 2.2: Kvadranty technického dluhu (Team Asana, 2025)

Úmyslný dluh vzniká vědomým rozhodnutím upřednostnit rychlost dodání před správným návrhem. Naproti tomu neúmyslný dluh bývá důsledkem nezkušenosti nebo nedostatku znalostí v době tvorby architektury. Osa obezřetnosti pak odráží schopnost týmu zhodnotit rizika při zavádění technického dluhu: zatímco obezřetný dluh je přijat s plným vědomím budoucích nákladů na jeho splacení, nedbalý dluh vzniká s nedostatkem informací či zkušeností (Fowler, 2009).

2.1.2 Nešváry

Pokud je technický dluh podle mé dřívější definice jev způsobující celkovou nepřehlednost a neudržitelnost systému, vyvstává rozumná otázka - jak konkrétně technický dluh vypadá? Technický dluh je tvořen hromaděním kusů kódu, které degradují kvalitu softwarového řešení. Tyto kusy se nazývají *code smells*, a v této práci je nazývám **nešváry**.

Před samotným výčtem důležitých nešvárů je vhodné se pozastavit nad jejich tříděním a podívat se, jakým způsobem je odborníci klasifikují. Pro účely této práce je důležité zmínit dvě klasifikace, které na problém nahlíží zcela odlišně a přivádí do problematiky důležité myšlenky - Mäntylovu klasifikovaci a Perezovu klasifikaci. Mäntylä nahlíží na nešváry jako na skupiny příbuzných symptomů. Dívá se tedy na nešváry optikou jejich dopadu na kód samotný. Jednotlivé kategorie definoval na základě společného negativního dopadu na kvalitu kódu (Mäntylä, 2010; Shvets, 2018a):

- **Nabobtnalci** - tedy kód, který se rozrostl do takových proporcí, že s ním nelze nadále efektivně pracovat;
- **Zneužívači** - tedy kód, který špatně využívá možnosti objektového návrhu;
- **Zabraňovači** - tedy kód, který ztěžuje změny či další vývoj;
- **Zbytečníci** - tedy zbytečné části kódu, které ztěžují jeho pochopení;
- a **Vazbiči** - tedy případy, kdy je kód nesprávně provázaný a dochází ke špatnému rozdělení odpovědnosti.

Alternativní klasifikaci nabízí právě Perez, který netřídí nešváry do skupin podle společných vlastností, ale přišel s myšlenkou, že tyto nešváry tvoří de-facto monoid s operací skládání. Perez totiž popisuje některé nešváry jako nízkoúrovňové a atomické, tudíž jsou ireducibilní a tvoří základní kámeny pro skládání složitějších nešvárů, které nazývá vysokoúrovňovými (Perez, 2011). Tento pohled na nešváry závádí do problematiky důležitou myšlenku - že nešváry nejsou disjunktními problémy, ale že i v případě Mäntylovy klasifikace existuje míra překrytí.

Právě Pérezova myšlenka hierarchického skládání nešvarů vysvětluje, proč je v praxi obtížné stanovit mezi Mäntylovými kategoriemi ostré hranice. Atomické nešváry, jako je například *duplicitní kód*, totiž často fungují jako katalyzátory pro vznik složitějších, vysokoúrovňových problémů. Pokud se identická logika vyskytuje na mnoha místech systému, nevyhnutelně to vede ke *shotgun surgery*, kdy i triviální změna požadavku vynucuje koordinovanou netriviální změnu napříč rozptýlenými moduly.

Kromě této přímé posloupnosti, kdy jeden nešvar vyvolává druhý, však v kódu existují i vztahy, které připomínají spíše korelaci než jasnou kauzalitu. Typickým příkladem je právě *velká třída* (Large Class). Jakožto zástupce *nabobtnanců* má symptom mnoha řádků kódu, ale v těchto řádcích se mohou ukrývat další nešváry, které nemají s velkou třídou kromě kolokace nic společného. Perez by hledal přímou souvislost mezi těmito jevy, tudíž že všechny dohromady skládají velkou třídu. Mäntyla ale dovoluje uvažovat bezpříčinné překrytí kategorií a soustředit se na kvantitativní výčet.

Volba mezi těmito dvěma klasifikacemi pak v praxi závisí především na cíli samotné analýzy. Zatímco Pérezův přístup je nepostradatelný při snaze pochopit hloubkovou strukturu dluhu a identifikovat jeho primární příčiny, Mäntylova klasifikace nabízí přímočařejší cestu ke kvantitativnímu výčtu symptomů. Pro vývojáře je v danou chvíli důležité se rozhodnout, zda se zaměří na sledování kauzálních řetězců a bude se snažit eliminovat dluh u jeho kořene, nebo zda využije horizontální překryv kategorií k rychlé identifikaci nejvíce degradovaných modulů.

V následující tabulce 2.1 jsem vybral nešváry, které považuji za stěžejní pro praktickou část této práce:

Tabulka 2.1: Výběr nešvarů, jejich symptonů a příčin

Název	Symptomy	Příčiny	Možné řešení
Dlouhá metoda	Metody nad 20 řádků.	Nedodržení principu jedné odpovědnosti	Rozdělení na více metod
Velká třída	Třída, která dělá více věcí zároveň	Nedodržení principu jedné odpovědnosti.	Rozdělení na více tříd
Divergentní změna	Mnoho úprav v jednom souboru.	Kopírování kódu.	Vytvoření třídy
Shotgun Surgery	Jedna úprava v mnoha souborech.	Jedna odpovědnost v mnoha třídách.	Vytvoření třídy
Duplicitní kód	Stejná logika na více místech.	Kopírování kódu.	Vytvoření metody
Mrtvý kód	Nevyužité metody/proměnné.	Strach ze smazání kódu.	Odstranění kódu
Switch Statements	Rozsáhlé switch/if struktury.	Neobratné využití OOP.	Polymorfismus
Magické literály	Čísla a řetězce v řídících strukturách	Chybějící abstrakce, snaha o rychlé řešení	Nahrazení symbolickou konstantou

2.1.3 Detekce technického dluhu

Každý nešvár přispívá k celkovému technickému dluhu. Nešváry jsou příčinou, technický dluh stavem a netrivialita změn následkem. V důsledku se tak technický dluh v projektu projevuje jako překážka v životním cyklu softwaru, kterou je nutné nejprve identifikovat, zmapovat a zpětně dotrasovat k nešvárům.

Prvotním indikátorem přítomnosti technického dluhu je obvykle dynamika samotného vývoje. Pokud se rychlost vývoje snižuje a čas potřebný k implementaci nových funkcionalit či opravě zdánlivě prostých chyb roste, bývá jednou z příčin právě nutnost vývojářů neustále překonávat bariéry v podobě zanedbaného či nepřehledného kódu. Právě samotní vývojáři nabízí sice subjektivní, avšak kriticky důležité hodnocení. Ti jsou totiž schopni přesně identifikovat konkrétní úseky systému, jež je při práci nejvíce zpomalují.

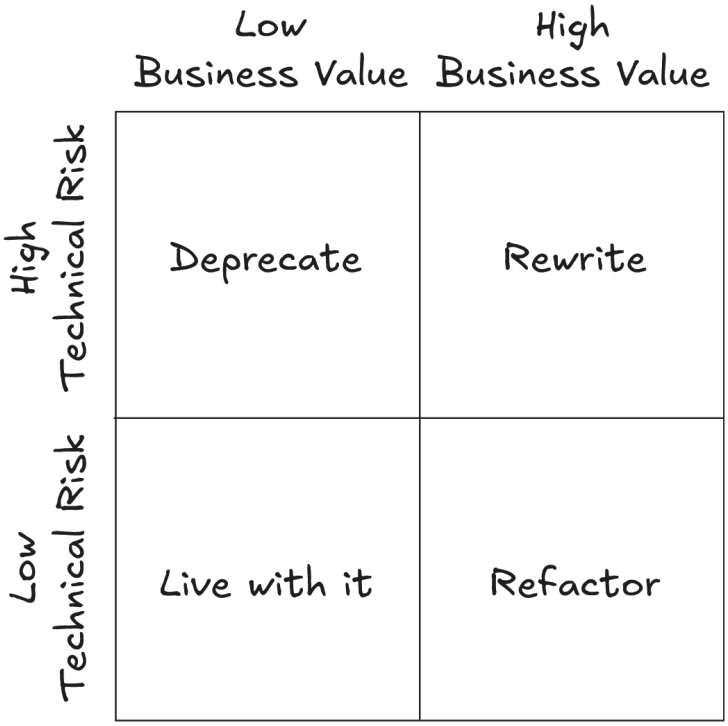
Objektivnější pohled nabízejí nástroje pro statickou analýzu kódu, které dokáží automatizovaně detekovat duplicity nebo antipatterny. Významnou metriku představuje rovněž code churn – ukazatel četnosti změn v konkrétních částech kódu. Vysoká míra netriviálních změn v určitém modulu často signalizuje jeho nevhodný návrh, nestabilitu nebo přílišnou provázanost s ostatními částmi systému, což jej činí náchylným k chybám a náročným na údržbu (Team Asana, 2025).

2.1.4 Řešení technického dluhu

Volba strategie pro řešení technického dluhu vyžaduje analýzu rozsahu, souvisejících rizik a dostupných zdrojů. Pro hodnocení lze využít standardních postupů jako analýza SWOT, výpočet poměru nákladů na vývoj a na opravu (*Technical Debt Ratio*) a nebo pokročilejších analýz pro hodnocení rizik, které si lze přizpůsobit pro účely hodnocení technického dluhu. (SonarSource, 2025)

Základní metodu pro vyhodnocení strategie splácení technického dluhu představuje rozhodovací matice, která dělí systémy do čtyř kategorií na základě dvou os: přínos pro podnik a technické riziko. Právě přínos pro podnik určuje, zda má smysl do daného systému investovat zdroje – pokud je vysoký, nemůžeme si dovolit nechat dluh nekontrolovaně hromadit a musíme zvolit cestu nápravy.

O konkrétním postupu pak rozhoduje osa technického rizika, která posuzuje pravděpodobnost selhání a závažnost jeho dopadů. V situaci, kdy je kód relativně stabilní nebo jsou následky případné chyby přijatelné, se jako ideální cesta jeví postupná refaktorizace. Ta umožňuje kontinuální splácení dluhu za plného provozu, čímž minimalizuje rizika regrese. Naopak v extrémním případě, kdy je systém pro podnik kritický, ale zároveň vykazuje neúnosnou míru rizika, se kompletní přepis stává nevyhnutelným. Přepis tak v tomto kontextu nepředstavuje selhání údržby, ale strategické rozhodnutí o vybudování nového, udržitelného základu.



Obrázek 2.3: Kvadranty řešení technického dluhu (ForrestKnight, 2025)

2.2 CLEAN CODE

Každý pokus o odstranění technického dluhu v praxi dříve či později naráží na nutnost provést v kódu netriviální změnu. Jak bylo definováno v kapitole 2.1, taková operace vyžaduje nejen hluboké pochopení stávající logiky a řídicích struktur, ale také značnou dávku kognitivního úsilí. Míra této zátěže však není dána pouze složitostí samotného problému, ale především kvalitou a stylem, jakým byl kód napsán.

Pro systematické snižování této kognitivní zátěže se v softwarovém inženýrství ustálil koncept *Clean Code*. Jedná se o soubor principů a doporučení, jejichž primárním cílem je zjednodušit čtení a údržbu zdrojového kódu. Aplikace těchto zásad v praxi přímo koreluje s kognitivní zátěží vývojáře: čím je kód „čistší“, tím méně mentálních kapacit musí věnovat jeho dešifrování a tím více prostoru zbývá na samotnou refaktorizaci a tvorbu nových funkcionalit (Martin; Coplien, 2009).

Clean Code není pouhým souborem pravidel, ale především způsobem uvažování o kódu jako o komunikačním prostředku. Jeho cílem je sjednotit sdělení záměru autora a zjednodušit budoucí údržbu či rozšiřování systému. Jak uvádí Robert Martin, jednotlivé zásady nelze vnímat jako absolutní dogmata; jejich konkrétní aplikace je vždy výsledkem odborné diskuse v rámci týmu (Martin; Coplien, 2009).

2.2.1 Význam a záměr identifikátorů

Pokud rozvineme myšlenku, že kód je komunikačním prostředkem, pak nejdůležitějším nástrojem této výměny informací je právě volba jmen. Nevhodné pojmenování je totiž nejčastější formou nízkoúrovňového technického dluhu – je levné jej vytvořit, ale jeho kumulativní úrok v podobě kognitivního úsilí při každém dalším čtení je enormní.

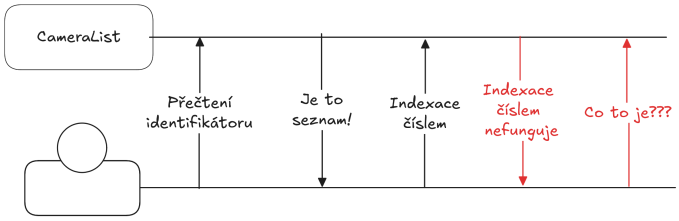
Základním stavebním kamenem čitelnosti je požadavek, aby každý název nesl jasný význam a záměr. Identifikátor by měl bez nutnosti nahlížet do implementace odpovídat na otázky:

- Proč tato entita existuje?
- Co tato entita dělá?
- Jakým způsobem se entita používá?

Zatímco jednopísmenný či zkratkovitý název proměnné je pro čtenáře pouze šumem, název jako `delkaPreponyCm` v sobě integruje informaci o fyzikálním významu i jednotkách, čímž okamžitě snižuje mentální náročnost pochopení daného výpočtu (Martin; Coplien, 2009).

2.2.2 Pravdivost a jednoznačnost identifikátorů

Kromě významu a záměru identifikátoru musíme dbát také na jeho integritu – název zkrátka nesmí lhát. Pokud kód obsahuje dezinformace, dochází k rozporu mezi účelem a skutečnou implementací. Označení proměnné jako `seznamUzivatelu`, přestože se ve skutečnosti jedná o slovník, vnáší do kódu zavádějící intuici. Vývojář na základě názvu podvědomě očekává vlastnosti seznamu, jako je garantované pořadí nebo indexace přirozenými čísly. Jakmile je tento předpoklad při ladění programu vyvrácen, dochází k okamžité ztrátě důvěry, neboť každá další konstrukce může také lhát.

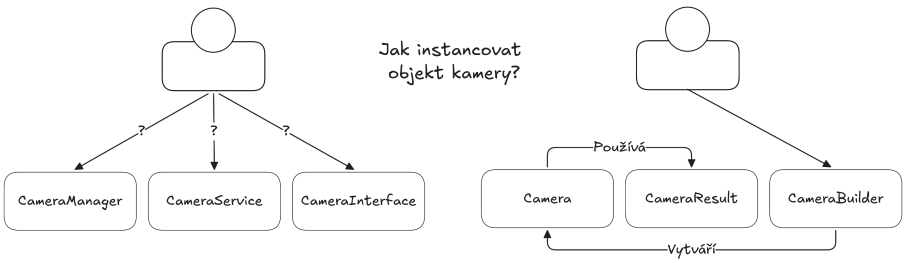


Obrázek 2.4: Nepravdivé názvy vytvářejí překážky ve vývoji

Stejně rizikové jsou i příliš podobné názvy. V éře moderních IDE, která využívají našeptávání kódu či AI asistenty, může minimální rozdíl mezi dvěma identifikátory vést k těžko odhalitelným chybám. Tento problém se navíc stupňuje s rostoucí délkou identifikátorů. Snaha o extrémní popisnost může paradoxně vést k tomu, že se podstatná informace ztratí v záplavě technického šumu (Martin; Coplien, 2009).

2.2.3 Sémantické odlišení identifikátorů

V návaznosti na jednoznačnost identifikátorů je nutné zohlednit také jejich sémantické odlišení. Pokud jsou identifikátory jednoznačné, ale odlišené jsou pouze pomocí sufixů (např. `data1`, `data2`) nebo vágních afixů jako `manager` či `info`, je vývojář odkázán na vlastní interpretaci vágního názvu nebo na studium implementace. Taková jména vznikají většinou v momentě, kdy vývojář rezignuje na hledání sémantického rozdílu mezi dvěma entitami. Pokud v systému existují dvě různé proměnné, musí existovat i sémantický důvod pro jejich odlišení, který by se měl promítnout do jejich názvu.



Obrázek 2.5: Semantické identifikátory implikující vazby mezi objekty

2.2.4 Hledatelnost a verbální komunikace

Kód není určen pouze pro kompilaci, ale i pro komunikaci a diskuzi uvnitř týmu. Identifikátory by proto měly být snadno vyslovitelné, aby se předešlo komickému, ale neefektivnímu hláskování nesrozumitelných zkratek. S tím souvisí i vyhledatelnost; unikátní a popisný název umožňuje vývojáři okamžitou orientaci v projektu. V tomto kontextu je kritické nahrazovat tzv. „magické konstanty“ pojmenovanými konstantami, které do řídicích struktur vnášejí potřebnou úroveň abstrakce (Martin; Coplien, 2009).

2.2.5 Očištění od technického šumu

Clean Code rovněž varuje před kódováním technických informací přímo do názvů. V éře silné statické analýzy je redundantní uvádět v názvu datový typ nebo prefixovat rozhraní písmenem `I`. Případná kolize názvu rozhraní a třídy (např. `Camera`) by neměla být řešena technickým prefixem `ICamera`, ale raději sémantickým upřesněním konkrétní role – například `CameraService` nebo `PhysicalCamera`. Takový přístup vkládá do názvu informaci o principu fungování, nikoliv pouze o syntaktické kategorii (Martin; Coplien, 2009).

2.2.6 Vlastnosti funkcí a metod

Zatímco vhodná jména tvoří slovní zásobu, funkce a procedury představují jeho gramatiku. Jsou první skutečnou úrovní abstrakce a základním nástrojem pro dekompozici složité logiky. *Clean Code* proto definuje tři pilíře pro jejich návrh:

- funkce by měla být krátká,
- měla by vykonávat právě jednu činnost
- a její působení musí být omezeno na vlastní kontext bez nechtěných *vedlejších účinků* (Martin; Coplien, 2009)

2.2.7 Délka a hloubka zanoření

Požadavek na minimalizaci délky funkce není samoúčelný. Jedná se především o obranu před kognitivním přetížením. Příliš dlouhá metoda s vysokou mírou zanoření v řídicích strukturách (cykly, podmínky) nutí vývojáře udržet v hlavě příliš mnoho stavových proměnných najednou. Jak trefně poznamenal Linus Torvalds: *„if you need more than 3 levels of indentation, you're screwed anyway, and should fix your program.“* (Torvalds, 2007). Každá úroveň zanoření navíc představuje ideální místo pro extrakci do samostatné, pojmenované metody s jasnou zodpovědností.

2.2.8 Jednotná úroveň abstrakce

Princip jediné činnosti vyžaduje, aby si funkce zachovala konzistentní úroveň abstrakce. Funkce by měla buď vykonávat příkaz, nebo odpovídat na dotaz, ale nikdy obojí současně (princip *Command Query Separation*). Strategická logika by neměla být mísena s nízkoúrovňovými detaily, jako je formátování řetězců nebo výpočty. Tento hierarchický přístup lze přirovnat k fungování firmy: ředitel (vysokoúrovňová funkce) vydává strategické pokyny, zatímco operativní detaily deleguje na vedoucí skladů, kteří řídí konkrétní naskladnění zboží (Martin; Coplien, 2009).

2.2.9 Pravidlo sestupného čtení

Pro udržení této hierarchie abstrakce je doporučeno dodržovat pravidlo sestupného čtení (*The Stepdown Rule*). Program by měl být čitelný jako plynulé vyprávění, kde každý odstavec rozvíjí detaily předchozího. V kontextu objektového návrhu to znamená, že implementační detaily následují až po metodách, které je volají (Martin; Coplien, 2009):

- K vyřízení objednávky je třeba zabalit zboží, odečíst jej ze skladu a vytisknout štítek.
- K zabalení zboží je třeba položku vyhledat a zvolit vhodný obal.
- K odečtení ze skladu je třeba identifikovat pozici zboží v registru.

2.2.10 Rozhraní a počet argumentů

Komplexitu funkce však neurčuje pouze její tělo, ale také šířka jejího rozhraní. Ideální funkce je niladická (bez argumentů). Monadické a dyadické funkce jsou běžné a přijatelné, ale s každým dalším parametrem roste obtížnost pochopení záměru a dramaticky se komplikuje testování.

Uvažujeme-li funkci se čtyřmi parametry typu *boolean*, vzniká celkem 2^4 možných kombinací vstupních stavů, které je v ideálním případě nutné validovat. Obecně je pak počet všech teoreticky možných testovacích případů roven mohutnosti kartézského součinu množin přípustných hodnot všech jednotlivých argumentů. Pro k argumentů stejného výčtového typu o n prvcích je tento počet dán vztahem n^k. U spojitých proměnných je situace ještě kritičtější, neboť vyžaduje pečlivý výběr reprezentativních zástupců z tzv. tříd ekvivalence, aby bylo možné pokrýt hraniční stavy bez nekonečného nárůstu testovacích scénářů (Martin; Coplien, 2009).

Vysoký počet argumentů navíc smazává přirozené pořadí parametrů, což zvyšuje riziko jejich záměny při volání – výjimkou jsou pouze případy kdy mezi parametry již existuje přirozené uspořádání, jako jsou například souřadnice v kartézské soustavě (x, y). (Martin; Coplien, 2009)

2.2.11 Seskupování argumentů a příznaky

Pokud funkce vyžaduje velký počet parametrů, je to jasný signál, že některé z nich tvoří samostatný koncept a měly by být seskupeny do logické datové struktury či objektu. *Clean Code* rovněž důrazně varuje před používáním příznakových argumentů (*flag arguments*). Pokud funkce přijímá *boolean*, který větví její tok, z definice vykonává více než jednu věc a měla by být rozdělena do samostatných, jasně pojmenovaných funkcí. (Martin; Coplien, 2009).

2.3 NÁVRHOVÉ VZORY

Zatímco *Clean Code* se soustředí na disciplínu v nejnižší úrovni abstrakce – tedy na to, jak psát čitelný a srozumitelný kód v rámci funkcí a tříd. Návrhové vzory posouvají tuto snahu o řád výše. Představují totiž konceptuální řešení častých problémů v návrhu softwarové architektury. Na rozdíl od algoritmu, který definuje striktní sekvenci kroků k řešení problému, jsou návrhové vzory spíše obecnými šablonami či strategiemi. Vývojář tyto strategie adaptuje v souladu se specifiky programovacího jazyka a potřebami daného systému (Shvets, 2018b). Vzory tak přenášejí žádoucí vlastnosti *Clean Code* do vyšších úrovní abstrakce, kde se formují rozsáhlejší architektonické celky. Skrze jejich použití definujeme nejen záměr a myšlenku za existencí jednotlivých komponent, ale i jejich očekávané chování a vzájemné interakce. Rozpoznání vzoru v kódu tak vývojáři okamžitě odhaluje sémantiku celé komponenty bez nutnosti studovat detailní implementaci.

2.3.1 Specifikace návrhových vzorů

Popis samotných návrhových vzorů se v praxi pohybuje na různých úrovních formality. Nejčastěji se setkáváme s neformálními popisy, které kombinují text a grafické notace, jako jsou blokové diagramy nebo UML diagramy (Shvets, 2018b). Tyto vizualizace jsou sice dobrou reprezentací návrhového vzoru, ale neposkytují dostatečně pevný základ pro kontrolu vlastností navrženého systému. Pokud tedy chceme analyzovat kód a hledat místa pro využití návrhových vzorů, vzniká otázka, jak je alespoň kvaziformálně definovat a porovnávat.

Formalismy nejsou přitom softwarovému oboru cizí. V oblasti návrhu a teorie databází slouží k popisu manipulace s daty relační algebra a k definici logických podmínek relační kalkul. I v objektovém návrhu vznikaly snahy o zavedení formálního aparátu, který by dokázal modelovat vztahy a komunikaci mezi objekty při zachování klíčových vlastností OOP, jako je zapouzdření nebo schopnost kompozice. Cílem těchto pokusů bylo tedy vytvořit tzv. objektový kalkul.

Nierstraszova notace

Významným příspěvkem k formalizaci objektového návrhu byla práce Oscara Nierstrasze, který pro tento účel propojil lambda kalkul s procesním kalkulem. Volba těchto dvou disciplín byla strategická: lambda kalkul použil k definici vnitřních stavů a chování objektů, zatímco procesní kalkul použil pro modelování vzájemné komunikace objektů. Nierstrasz tak využil silné stránky dvou etablovaných disciplín a spojil je do jednoho, zdánlivě stejně silného, aparátu, který nazývám *Nierstraszova notace* (Nierstrasz, 1992).

Ve své notaci Nierstrasz představil koncept agentů, které reprezentují objekty. Agenti jsou uspořádanou n-ticí definovánou třemi množinami: jména, vstupy a výstupy. Mezi těmito agenty následně vytvořil řadu operátorů, například pro modelování rekurze, kompozice, aplikace funkcí a dalších prvků objektového i funkcionálního programování, které jsou ukázány v rovnici 2.1 (Nierstrasz, 1992).

$$a ::= a\&a \mid n := a \mid a|a \mid x \rightarrow a \mid v \wedge a \mid a@v \tag{2.1}$$

Konkrétní práci s Nierstraszovou notací lze ukázat na definici prázdného zásobníku: $empty := (push, x?) \rightarrow stack@(x, empty)$. Tento zápis popisuje objekt, který disponuje operací *push* s lokálním argumentem x. V momentě, kdy objekt přijme zprávu s touto signaturou, transformuje se na instanci *stack*, které jsou předány parametry v podobě přijatého argumentu a reference na původní objekt *empty*. Logika této transformace vychází z definice zásobníku v rovnicích 2.2 a 2.3, kde je *stack* koncipován jako entita přijímající parametry *top* (aktuální hodnota x) a *rest* (předchozí stav zásobníku) (Nierstrasz, 1992).

$$stack := (top?, rest?) \rightarrow (pop, top) \wedge rest \tag{2.2}$$

$$stack := (top?, rest?) \rightarrow (push, x?) \rightarrow stack@\,(x, stack@(top, rest)) \tag{2.3}$$

Z rovnice 2.2 je vidět, že operace *pop* vrací hodnotu *top* a zároveň obnovuje stav *rest*, čímž je realizována struktura LIFO. Naopak rovnice 2.3 ukazuje rekurzivní povahu operace *push*, která vytváří novou instanci zásobníku zapouzdřující stav té předchozí.

Ačkoliv Nierstrasz skvěle modeluje předávání zpráv, z pohledu detekce návrhových vzorů je příliš nepoužitelný. Nutí nás sledovat každou jednotlivou zprávu a transformaci agenta, abychom pochopili, že se jedná i o obyčejný zásobník. S rostoucí složitostí systému poroste i náročnost popisu rozsáhlých architektur.

Zásadním nedostatkem je z mého pohledu také absence explicitního vyjádření vazeb mezi objekty. Nierstraszova notace se soustředí primárně na sekvenci zpráv, čímž v podstatě vytváří formální verzi sekvenčního diagramu.

Mikkonenova notace

Na problém vyjádření vazeb již netrpí Mikkonenova notace, která je založena na metodě DisCo. Mikkonen se odklání od popisu komunikace pomocí zpráv a soustředí se na objekty jako na součásti širšího celku, přičemž klade důraz na jejich vzájemné vztahy. Jednotlivé třídy definuje jako množiny atributů a relací, přičemž relace mají explicitně vyjádřenou kardinalitu. Obecný zápis takové relace uvádí rovnice 2.4, kde je vztah definován jako výběr z kartézského součinu dvou tříd (Mikkonen, 1998).

$$(n) \cdot R \cdot (m) : C \times D \tag{2.4}$$

Konkrétní příklad relace ukazuje rovnice 2.5 na relaci *Attached* mezi třídami *Subject* a *Observer*. Tento zápis definuje, že k libovolnému počtu instancí třídy *Observer* může být připojen buď jeden konkrétní *Subject*, nebo žádný (vazba neexistuje). Relace tedy umožňuje, aby se jeden *Subject* registroval u více pozorovatelů, zatímco každý *Observer* může nekonečně mnoho vazeb a jednotlivé instance třídy *Subject* (Mikkonen, 1998).

$$(0..1) \cdot Attached \cdot (*) : Subject \times Observer \tag{2.5}$$

Jednotlivé relace nevznikají svévolně; ačkoliv mohou být pevně definovány již při inicializaci systému, mnohdy se ustavují dynamicky v čase prostřednictvím společných akcí (joint actions). Akce je v tomto pojetí chápána jako funkce, do níž vstupuje více objektů současně. Každá taková operace disponuje seznamem účastníků, aktivační podmínkou a definicí následných změn stavů (Mikkonen, 1998).

$$A(c : C, i) : i \neq c.x \rightarrow c.x' = i \tag{2.6}$$

Příklad takové akce uvádí rovnice 2.6. Akce A přijímá jako parametry instanci c třídy C a proměnnou i. Pokud je splněna podmínka, že se hodnota i nerovná aktuálnímu atributu x objektu c, dojde k aktualizaci tohoto atributu na hodnotu i. V podstatě se jedná o formální model setteru třídy, respektive atomické přiřazení nové hodnoty atributu (Mikkonen, 1998).

Ačkoliv je Mikkonenova notace pro popis architektury a návrhových vzorů výrazně lepším nástrojem než Nierstraszova notace, stále zůstává příliš svázána s konkrétním vnitřním stavem komponent. Mikkonen například při formalizaci vzoru *Observer* operuje s pevně definovanými atributy subjektů, jako je *Data* (Mikkonen, 1998). I když jsou samotné vazby popsány skvěle, tento přežitek vnitřního stavu v praxi komplikuje srovnání ideální struktury vzoru s reálným zdrojovým kódem. V praktickém nasazení je totiž téměř nemožné dokázat shodu formálního popisu s existujícím řešením, pokud se implementační detaily (názvy a počty atributů) byť jen minimálně odchylují od vzoru.

Hybridní notace

Právě dříve zmíněné nedostatky obou notací mě zavedly na cestu snahy o vlastní způsob formalizace návrhových vzorů. Cílem bylo nalézt rigorózní základ, který by nebyl zkostnatělý a nabídl by dostatečnou flexibilitu pro popis struktury i dynamických vztahů. Prvotním impulzem byl pokus o syntézu Nierstraszovy a Mikkonenovy notace – snaha získat z obou modelů jejich nejsilnější stránky, podobně jako Nierstrasz kdysi sjednotil lambda kalkul s procesním kalkulem. Příklad této syntézy je vidět v rovnici 2.7.

$$Factory := (create, x?) \xrightarrow[Factory \times Product]{(0..1) \cdot Created \cdot (*)} Product@(x) \tag{2.7}$$

Brzy jsem však narazil na ty samé problémy, za které jsem Nierstraszovu notaci kritizoval. S každou další úpravou a pokusem o zpřesnění modelu jsem se ocital v situaci, kdy jsem se v jednotlivých iteracích přibližoval k cíli, ale za cenu neúměrného nárůstu komplexity a ztráty elegance celého zápisu.

Rozhodl jsem se proto udělat krok zpět, a podívat se na již známé matematické koncepty, které by mohly sloužit jako univerzální základ pro popis návrhových vzorů. Mé prvotní úvahy směřovaly k diskrétní matematice a abstraktní algebře, konkrétně k lineárním zobrazením. Ta sice dokáží efektivně modelovat vazby, jsou však stále příliš úzce asociována s funkcemi či množinami, což by mě limitovalo z hlediska aplikace. Potřeboval jsem najít způsob, jak jít na vyšší úroveň abstrakce.

2.3.2 Specifikace pomocí teorie kategorií

Pokračoval jsem tedy následující myšlenkou – proč nejít na tu nejvyšší úroveň matematické abstrakce, kterou znám. To mi dovolí soustředit se na volné modelování návrhových vzorů bez toho, aniž bych je podroboval formálním omezením. Naopak, pokud budu potřebovat řešit například kardinalitu, mohu tato omezení dodatečně doplnit do definice.

V minulosti jsem se okrajově zajímal o teorii kategorií, a tak jsem začal s experimentálním popisem vybraných vzorů právě pomocí kategoriálních struktur. Má očekávání byla nanejvýš předčena. Teorie kategorií je dostatečné flexibilním nástrojem nejen k popisu návrhových vzorů, ale i k jejich vzájemnému porovnávání. Rozhodl jsem se proto v této myšlenkové linii pokračovat a návrhové vzory tímto způsobem formalizovat.

Je však nutné zdůraznit, že tato práce není zaměřena na matematickou podstatu teorie kategorií, a ani si neklade za cíl její hloubkový teoretický rozvoj. Cílem je využít základní principy a abstrakce, které tato disciplína zavádí, a aplikovat je na modelování softwarových struktur. Před samotným užitím teorie kategorií je třeba rozvést, co je vůbec jejím předmětem - kategorie, morfismy a funktory.

Kategorie

Kategorie $\mathcal{C}$ je definována jako struktura tvořená kolekcí objektů $\mathcal{O}$ a kolekcí morfismů $\mathcal{M}$, přičemž nad kategorií je definována operace skládání ($\circ$) a pro každý objekt existuje morfismus identity. (Ma, 2026)

$$\mathcal{C} = (\mathcal{O}, \mathcal{H}, \circ, \mathrm{id}) \tag{2.8}$$

V kontextu teorie kategorií se pro seskupení objektů používá termín kolekce záměrně místo více známé množiny. Zatímco množina vyžadují striktní dodržování formálních pravidel a axiomů teorie množin, termín kolekce funguje jako sémanticky volnější seskupení prvků, které nemusí tvořit rigorózně definovanou strukturu. Použití kolekce nám tak rozvazuje ruce – umožňuje efektivně seskupovat objekty a morfismy i v případech, kdy by striktní aplikace teorie množin vedla k logickým rozporům. Posledním rozšířením kolekce je tzv. *rodina objektů*. Jedná se o kolekci, ve které existuje zobrazení mezi prvky kolekce a kategorií přirozených čísel, čímž se efektivně stává indexovanou. (Ma, 2026)

Objekty

Objekty jsou reprezentací předmětů zkoumání - funkce, prostory, metriky, a v našem případě softwarové třídy a funkční jednotky. Samotné objekty ovšem mohou reprezentovat i celé kategorie, čímž je umožněno vytvářet podkategorie a definovat pro ně speciální pravidla v rámci většího celku. Objekty značím velkými písmeny latinky. (Ma, 2026)

Morfismy

Morfismy jsou zobrazením mezi objekty, které transformují jeden objekt z definičního oboru na objekt z oboru hodnot. Slouží jako nositelé vazeb, transformací či operací. Symbolizují tok informací a jak se postupně objekty mění. Morfismy značím malými písmeny latinky. (Ma, 2026)

Diskrétní kategorie

Podobně jako můžeme množiny prvků rozložit na sjednocení jednoprvkových množin, či permutační cykly rozložit na součin transpozic, můžeme i kolekce objektů rozložit na jednoprvkové podkolekce a utvořit z nich samostatné kategorie. Tyto jednoobjektové podkategorie nazýváme diskrétními kategoriemi. Jejich význam je nesmírný: pokud totiž přestaneme tyto izolované objekty vnímat jako pouhé prvky kolekce a vybavíme je vnitřní strukturou, transformujeme je na monoidy. Monoidy mají kolekce morfismů nad svým jediným objektem (reprezentující jednotlivé interakce či operace objektu), které lze díky binární operaci skládání libovolně řetězit. Existence identity pak v tomto systému představuje morfismus, který zachovává objekt beze změny. (Ma, 2026)

Homomorfismy

Pokud máme prvky kolekce, které si s sebou nesou vlastní strukturu (monoidy), musíme také patřičně upravit morfismy, kterými je transformujeme. V obecné kategorii bez dodatečných pravidel by morfismus mohl s prvky objektu nakládat libovolně, což by v našem případě vedlo k sémantickému rozpadu: vnitřní operace a interakce objektu by po transformaci ztratily svůj význam.

Všechny kategorie v této práci budu dále definovat jako kolekci objektů a kolekci homomorfismů. V momentě, kdy objekty přestanou být beztvarými prvky a začneme je vnímat jako monoidy, musí se tomuto faktu přizpůsobit i vazby mezi nimi. Homomorfismus zde plní roli zobrazení, které strukturu objektů neignoruje, ale naopak ji přenáší a zachovává. Aby tohle platilo, musí být splněna podmínka zachování operace (rovnice 2.10) a zachování identity (rovnice 2.11)

$$(S, \circ, id_S) \text{ a } (T, \bullet, id_T) \tag{2.9}$$

$$\forall x, y \in \mathcal{M} : f(x \circ y) = f(x) \bullet f(y) \tag{2.10}$$

$$f(id_S) = id_T \tag{2.11}$$

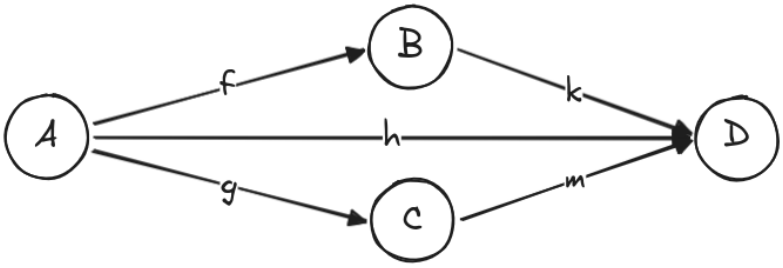
Udělejme krok zpět. Tento formální zápis v podstatě definuje pravidla překladu mezi různými úrovněmi softwarové abstrakce. Monoidy nepředstavují jen abstraktní množiny, ale konkrétní nositele logiky – například třídy, komponenty nebo rozhraní. Logiku lze reprezentovat dílčími morfismy uvnitř monoidu. Binární operace monoidů pak reprezentují skládání interakcí, tedy způsob, jakým na sebe v programu navazují jednotlivé metody nebo transformace stavu.

Zachování operace (rovnice 2.10) v praxi funguje jako pojistka proti rozpadu kauzality; Tato podmínka striktně vyžaduje, aby se systém při zřetězení operací choval přesně jako kdybychom je udělali samostatně a ručně spojili dohromady. Homomorfismus zde znemožňuje, aby se uvnitř objektu vytvořila de-facto nová logika, která se aktivuje pouze při určité kombinaci volání. V podstatě to garantuje, že kompozice metod v kódu nemá žádné skryté vedlejší efekty, které by nebyly definovány už v jejich jednotlivých obrazech.

Podmínka zachování identity (rovnice 2.11) pak řeší konzistenci výchozího stavu tím, že dohlíží na korektní přenos „nečinnosti“. Pokud v určitém kroku objekt nevykoná žádnou změnu stavu (operace typu No-Op, prázdná transformace nebo getter), homomorfismus zaručuje, že se tento neutrální prvek promítne i do navazujících vrstev systému. To v praxi eliminuje chyby, kdy volání zdánlivě „prázdné“ metody v jedné vrstvě nečekaně změní kontext v jiné vrstvě. Vyžadování homomorfismů nám tak dává jistotu, že architektura o fungování kódu nelže; nutí nás navrhovat vazby tak, aby výsledek složené operace byl vždy predikovatelný z jejích jednotlivých částí, bez ohledu na to, jak hluboko v hierarchii systému se nacházíme.

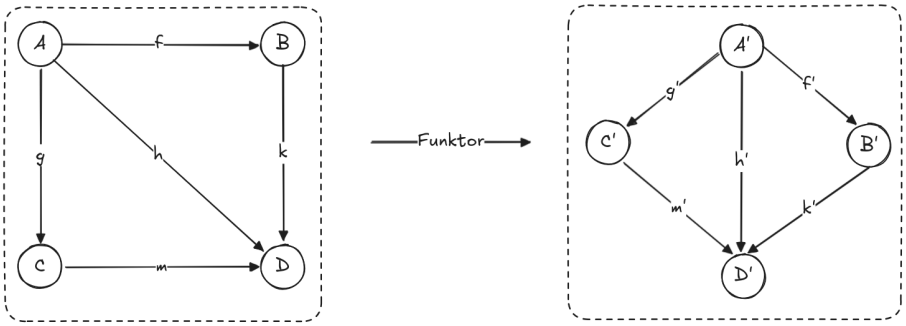
Komutativní diagramy

Kategorie, respektive jejich objekty a morfismy, se vizuálně zakreslují do diagramů, které strukturou připomínají orientované grafy. Diagram označujeme jako komutativní, pokud všechny cesty (posloupnosti morfismů) vedoucí ze stejného počátečního bodu A do stejného cílového bodu B představují identický výsledek. Komutativní vlastnost je důležitá, protože když není dodržena, můžeme se z bodu A dostat do bodu B, a pokaždé skončit s jiným výsledkem. Když diagram komutuje, máme zaručené, že výsledek bude vždy stejný. (Ma, 2026)



Obrázek 2.6: Komutativní diagram

Diagramy nám dovolují se soustředit na čistou strukturu kategorií. Často totiž zjistíme, že různé části systému – ačkoliv pracují s jinými daty – sdílejí identické uspořádání vazeb. Tato strukturální podobnost nám dovoluje definovat přechody kategoriemi pomocí funktoru. Funktor lze chápat jako vyšší typ zobrazení, který převádí objekty a morfismy jedné kategorie do druhé, aniž by narušil jejich vnitřní strukturu. (Ma, 2026)



Obrázek 2.7: Funktor

Pokud jsme si dříve definovali homomorfismus jako pojistku zachování struktury uvnitř monoidu, funktor je jeho analogií na úrovni celých kategorií. Zajišťuje, že pokud v původní kategorii existovala funkční cesta mezi body A a B, bude po transformaci funktorem existovat i v cílové kategorii a bude dávat stejný smysl. V praxi nám funktory umožňují například modelovat, jak se pravidla definovaná v návrhovém vzoru do konkrétní implementace, aniž by došlo k narušení logických vazeb mezi nimi.

Pokud návrhové vzory formalizuji jako kategorie a reprezentuji je pomocí komutativních diagramů, lze v kódu vyhledávat nejen přesné strukturání shody, ale také identifikovat částečné shody o různé vzdálenosti vůči ideálnímu vzoru či definovaným protivzorům. Pro tento účel je potřeba zavést několik relací mezi kategoriemi[1]:

- **Sémantická ekvivalence** ($A \approx B$) – Objekty jsou na sobě strukturálně nezávislé, ale sdílejí totožný koncepční záměr. V daném doménovém kontextu jsou logicky zaměnitelné. Platí $A \approx B : \mathcal{O}_A = \mathcal{O}_B \wedge \mathcal{H}_A \neq \mathcal{H}_B$
- **Izomorfismus** ($A \cong B$) – Objekty vykazují naprostou shodu v rozhraní i struktuře. Tato relace představuje ideální stav čistého návrhu, kdy jsou entity plně a bezpečně zaměnitelné bez vedlejších účinků na okolní systém. Platí $A \cong B : \mathcal{O}_A = \mathcal{O}_B \wedge \mathcal{H}_A = \mathcal{H}_B$
- **Kongruence** ($A \equiv B$) – Objekty sdílejí shodu v konkrétní části morfismů. Typicky se jedná o projev *duck typingu*; objekt sice disponuje očekávaným rozhraním, ale jeho vnitřní implementace či dodatečný balast narušují čistotu vzoru. Platí $A \equiv B : \mathcal{O}_A \subseteq \mathcal{O}_B \wedge \mathcal{H}_A \subseteq \mathcal{H}_B$. [2]
- **Strukturální podobnost** ($A \sim B$) – Objekty vykazují částečně shodné strukturní rysy (např. podobné grafové topologie), přestože mezi nimi neexistuje přímá vazba či společný předek. Platí $A \sim B \iff \exists U : Kategorie \to Graf \mid U(A) \cong U(B)$

Ačkoliv jsem u samotných definic relací ukázal platné vztahy, formálnější definici upozadím ve prospěch jejich širší aplikace. Důležité rozlišení mezi těmito relacemi je, že sémantická ekvivalence spojuje shodný záměr; izomorfismus shodné kategorie; kongruence shodu v rozhraní a strukturální podobnost shodu v topologické struktuře.

2.3.3 Popis vybraných návrhových vzorů

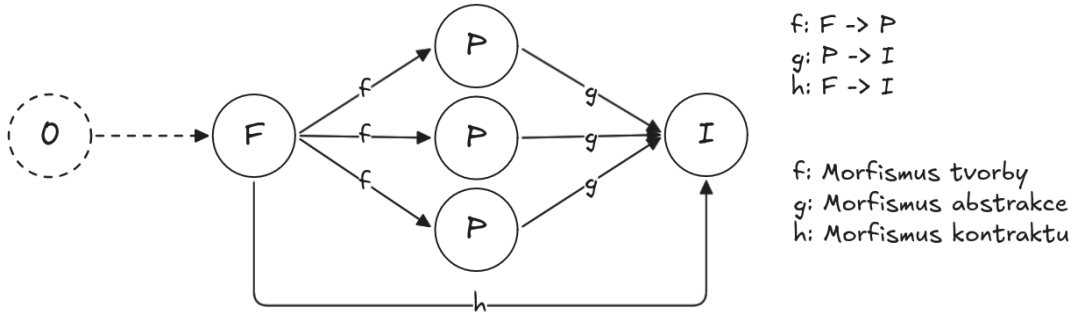
S teoretickým základem z předchozí podkapitoly je nyní možné aplikovat tyto koncepty na popis samotných návrhových vzorů. Není v možnostech této práce formalizovat všechny, proto jsem se rozhodl pro konkrétní vzory, se kterými jsem dobře obeznámen a které jsou hojně využity v praktické části.

[1] Je nutné poznamenat, že tyto relace se týkají i podkategorií. Není tak nutné porovnávat celou strukturu, respektive celý diagram, ale pouze jeho vybranou část.

[2] Kongruence je nekomutativní operací. Pokud je $A \equiv B$, pak musí platit $B \equiv A : \mathcal{O}_A \supseteq \mathcal{O}_B \wedge \mathcal{H}_A \supseteq \mathcal{H}_B$. V takovém případě ale vzniká riziko, že při záměně objektů narazíme na morfismus z $\mathcal{H}_A$, který není součástí množiny $\mathcal{H}_B$. Tím by se narušila integrita celé struktury.

Factory

Návrhový vzor *Factory* řeší nešváry z rodiny vazbičů a zabraňovačů. Umožňuje totiž vytvářet izomorfní třídy bez vzniku vazby na konkrétní implementaci. Tím, že *Factory* ve své podstatě vytváří jediný vstupní bod pro vznik těchto tříd, je nejenom snadné rozšířit systém o novou třídu, ale zároveň není nutné zasahovat do již existujícího kódu, který by jinak musel řešit logiku pro vytváření tříd.



Obrázek 2.8: Komutativní diagram návrhového vzoru: Factory

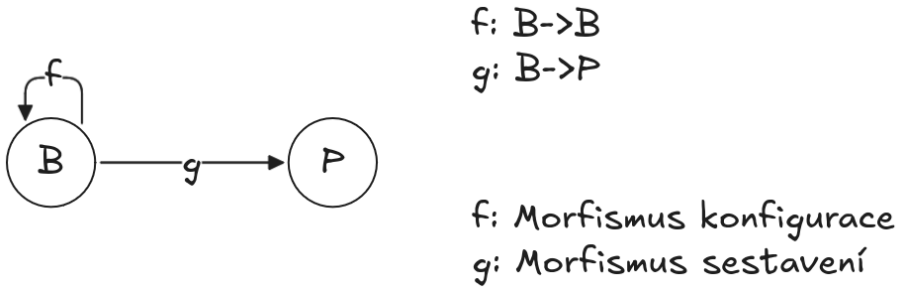
Formálně lze návrhový vzor *Factory* definovat jako kategorii tří objektů (*Factory, Product, Interface*) a tří morfismů (*tvorby, abstrakce a kontraktu*). Objekt *Factory* pomocí vnitřní logiky volí konkrétní *Product* k vytvoření, přičemž všechny tyto objekty musí odkazovat na, respektive implementovat, stejné rozhraní. Díky komutativitě diagramu a tranzitivitě skládání morfismů, dospějeme k závěru, že existuje morfismus kontraktu, který zaručuje, že objekt *Factory* vždy vytvoří objekt svázaný se společným rozhraním. Tento morfismus nejen že existuje implicitně, ale dokonce musí pro správnou funkci návrhového vzoru existovat explicitně. Pokud by totiž neexistoval, znamená to, že jeden z objektů *Produkt* neimplementuje společné rozhraní a nejsou tak navzájem kongruentní, natož izomorfmí.

Na základě výše uvedené formální definice lze provést analýzu SWOT pro vyhodnocení přínosů a rizik návrhové vzoru *Factory*:

- **Silné stránky** – Vzor je silný v situacích, kdy jsou sémanticky ekvivalentní objekty ($\approx$) v systému zaměňovány podle kontextu.
- **Slabé stránky** – Je pouze proxy pro izomorfní třídy, a nezabraňuje vývojáři obejít Factory přímo k implementaci. Navíc přidává vrstvu zanoření do zásobníku volání.
- **Příležitosti** – Příležitost tvoří třídy delegující činnost na více tříd bez sdíleného rozhraní.
- **Hrozby** – Aplikace vzoru u jednoúčelových tříd, kde neexistuje předpoklad budoucí existence alternativních implementací, je kontraproduktivní a zbytečnou abstrakcí.

Builder

Návrhový vzor *Builder* řeší nešváry převážně z třídy nabobtnalců a zneužívačů objektového návrhu. Dovoluje nám totiž komplexní objekt sestavit pomocí sekvence volání a reprezentovat tak širokou škálu různých konfigurací. Tímto přístupem se předchází vzniku nabobtnalých seznamů argumentů a zároveň se otevírá prostor pro definici vstupní logiky a validace u jednotlivých kroků. Builder tak transformuje proces tvorby objektu z jednorázového (často nepřehledného) úkonu na řízenou sekvenci operací.



Obrázek 2.9: Komutativní diagram návrhového vzoru: Builder

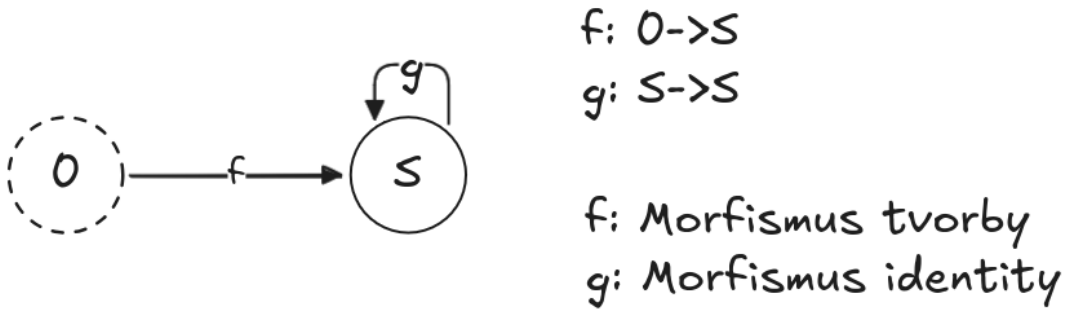
Formálně lze návrhový vzor *Builder* definovat jako kategorii sestávající ze dvou objektů (*Builder, Product*) a dvou morfismů (*Konfigurace, Sestavení*). Objekt *Builder* je inicializován a zároveň konfigurován pomocí automorfismu konfigurace, který postupně aktualizuje jeho vnitřní stav. Po dosažení požadovaného stavu je proces konfigurace dokončen sestavením, který transformuje zkonfigurovaný *Builder* na výsledný, zpravidla již imutabilní *Produkt*. Ačkoliv *Product* může být mutabilní, je pak těžké správně rozdělit odpovědnosti za jeho konfiguraci mezi *Builder* a *Product*. Objekt *Product* v tomto případě může být cokoliv od třídy po rozhraní, čímž umožňuje složit ho s dalšími návrhovými vzory.

Na základě výše uvedené formální definice lze provést analýzu SWOT pro vyhodnocení přínosů a rizik návrhové vzoru *Builder*:

- **Silné stránky** – *Builder* transformuje proces tvorby objektu z jednorázového úkonu na řízenou sekvenci operací.
- **Slabé stránky** – Proces konfigurace může být verbózní kvůli opakujícím se automorfismům konfigurace.
- **Příležitosti** – *Builder* se hodí v situacích, kdy funkce nebo metody mají dlouhé seznamy parametrů.
- **Hrozby** – Použití návrhového vzoru *Builder* nemusí být tolik vhodné jako použití *konfiguračního slovníku* nebo *Singletonu*.

Singleton

Návrhový vzor *Singleton* řeší nešváry převážně z třídy zneužívačů objektového návrhu. Má totiž jednoduchou, ale velmi důležitou funkci: zajistit existenci vždy jen jedné instance objektu v rámci celého programu. To se hodí například pro spojení s databází či sdílení globálního stavu.



Obrázek 2.10: Komutativní diagram návrhového vzoru: Singleton

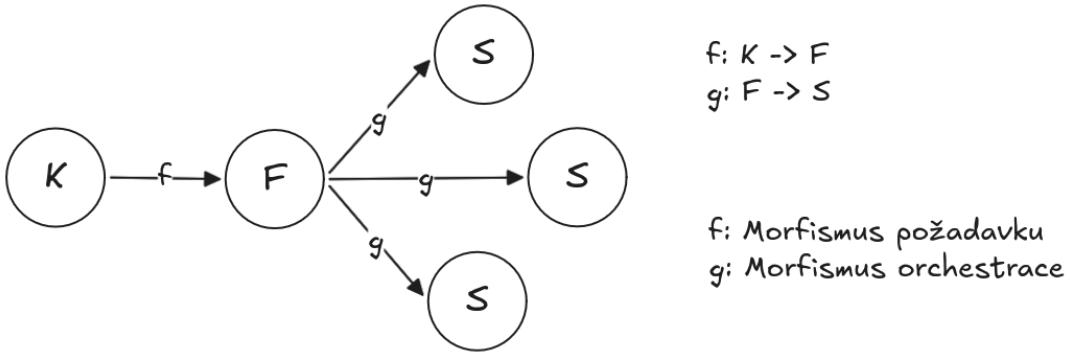
Formálně lze návrhový vzor *Singleton* definovat jako kategorii sestávající ze tří objektů (*Klient, Singleton a Produkt*) a tří morfismů, které definují mechanismus přístupu a správy instance. Prvním je morfismus přístupu, kterým *Klient* skrze veřejné rozhraní žádá o poskytnutí sdíleného zdroje. Na tento požadavek navazuje vnitřní automorfismus identity, který operuje nad stavem samotného *Singletonu*; jeho úlohou je ověřit existenci instance a zajistit, že opakovaná volání v čase budou vždy směřovat ke stejnému vnitřnímu stavu. Proces je završen morfismem zapouzdření, který mapuje *Singleton* na výsledný *Produkt*. Právě automorfismus identity zajišťuje správu instance produktu a garantuje, že *Klient* obdrží referenčně identický objekt bez možnosti ovlivnit mechanismus jeho vzniku či perzistence.

Na základě výše uvedené formální definice lze provést systematické vyhodnocení přínosů a rizik návrhové vzoru *Singleton*:

- **Silné stránky** – *Singleton* eliminuje potřebu předávat si referenci konkrétní instance; Sjednocuje přístup ke sdíleným zdrojům;
- **Slabé stránky** – Závislost dalších komponent na singletonu může zavést vedlejší účinky do dalších objektů.
- **Příležitosti** – Vhodné pro správu sdílených zdrojů a globálních proměnných, jako jsou magické hodnoty či konfigurace spojení s externími službami.
- **Hrozby** – Maskování špatného návrhu skrze sdílení proměnných, které by měly zůstat lokální.

Facade

Návrhový vzor *Facade* řeší nešváry převážně z tříd zbytečníků. Jeho cílem je totiž vytvořit jednodušší rozhraní pro komplexní podsystémy, které se mohou v systému objevit, a nad jejichž rozhraním nemusí mít vývojář kontrolu (například rozhraní externí knihovny).



Obrázek 2.11: Komutativní diagram návrhového vzoru: Facade

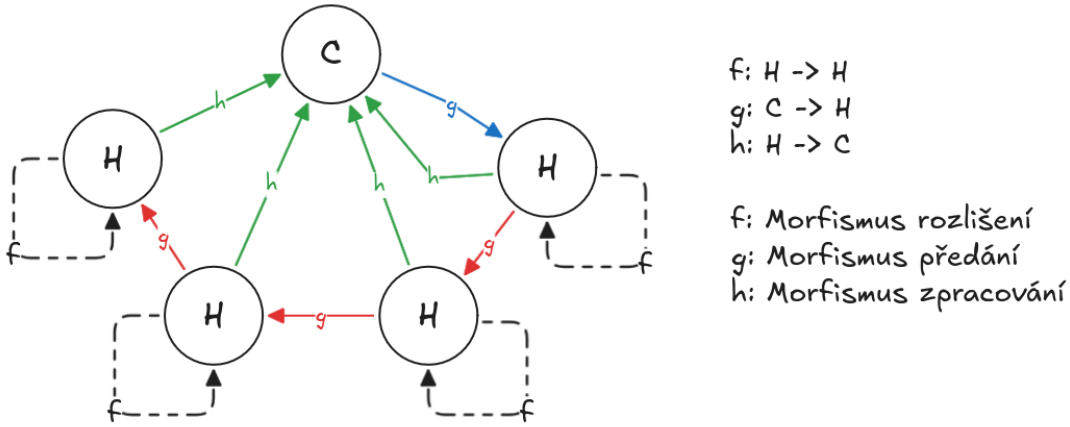
Formálně lze návrhový vzor *Facade* definovat jako kategorii ze tří objektů (*Klient, Facade a Subsystém*) a dvou morfismů, které reprezentují proces delegace a zjednodušení interakce. Prvním je morfismus požadavku, kterým *Klient* oslovuje sjednocené rozhraní objektu *Facade* s cílem vykonat komplexní operaci. Na tento podnět navazuje morfismus orchestrace, jenž operuje uvnitř *Subsystému*. Tento morfismus je ve skutečnosti složením mnoha dílčích transformací mezi vnitřními komponentami, které *Facade* koordinuje do jediného logického celku. V tomto modelu není podstatná transformace návratových hodnot, nýbrž samotná existence cesty, která vede od zjednodušeného rozhraní k neabstrahované logice. Komutativita diagramu je zde definována složením těchto dvou morfismů, což garantuje, že *Klient* dosáhne požadované funkcionality *Subsystému* výhradně skrze objekt *Facade*.

Na základě výše uvedené formální definice lze provést systematické vyhodnocení přínosů a rizik návrhové vzoru *Facade*:

- **Silné stránky** – Složitá, externí či nízkoúrovňová rozhraní, které nelze refaktorizovat, mohou být extrahovány do sémanticky smysluplných rozhraní.
- **Slabé stránky** – Závislost dalších komponent na singletonu může zavést vedlejší účinky do dalších objektů.
- **Příležitosti** – Zjednodušení rozhraní externích systémů a zapouzdření běžných operací v rámci domény.
- **Hrozby** – Předčasné zapouzdření do návrhového vzoru *Facade* místo refaktorizace, když je to možné. Nadbytečné užívání návrhového vzoru tam, kde to nemá z hlediska zlepšení sémantiky význam.

Chain of Responsibility

Návrhový vzor *Chain of Responsibility* (zkráceně: COR) řeší nešváry ze třídy vazbičů a nabobtnalců. Toho dosahuje rozkladem podmíněného zpracování do posloupnosti objektů, ve které každý objekt nese odpovědnost za specifický případ. Každý objekt tak kontroluje, jestli má daný případ zpracovat, nebo poslat dalšímu.



Obrázek 2.12: Komutativní diagram návrhového vzoru: COR

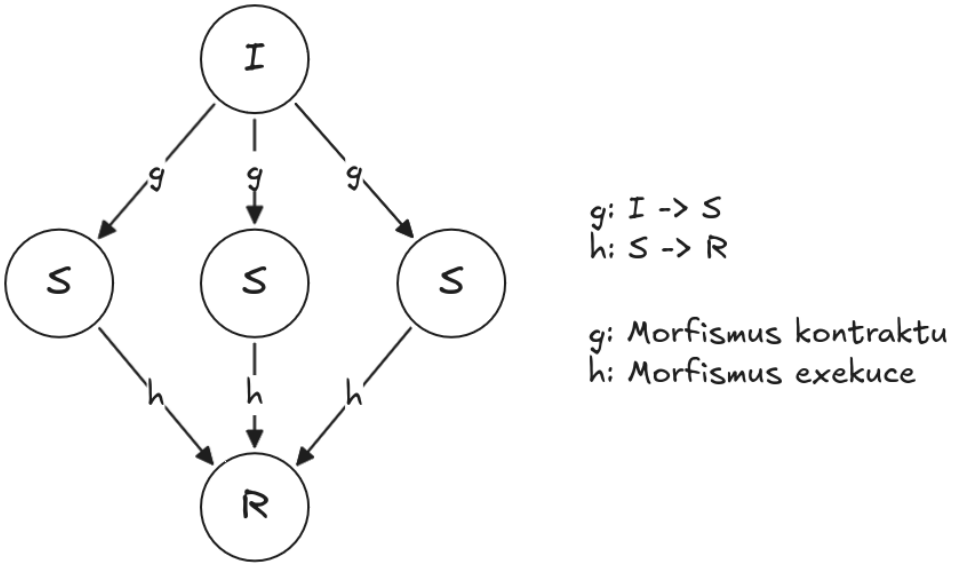
Formálně lze návrhový vzor *Chain of Responsibility* definovat jako kategorii ze dvou objektů (*Chain a Handler*) a tří morfismů, které definují mechanismus šíření a odbavení požadavku. Objekt *Chain* zahajuje proces a pomocí morfismu předání aktivuje první uzel v posloupnosti. Každý uzel následně aplikuje vnitřní morfismus rozlišení, kterým určí směr šíření požadavku v diagramu. Tento morfismus určí, zda uzel požadavek zpracuje pomocí stejnojmenného morfismu, nebo jej propustí dále do struktury pomocí morfismu předání. Proces předávání je zakončen terminálním uzlem, který z definice nesmí požadavek dále propouštět a musí jej zpracovat, čímž se zajišťuje konečnost celého procesu.

Na základě výše uvedené formální definice lze provést systematické vyhodnocení přínosů a rizik návrhové vzoru *Chain of Responsibility*:

- **Silné stránky** – Každý objekt zpracovává konkrétní situaci za konkrétně dané podmínky.
- **Slabé stránky** – Podmínky musí být správně navrženy, aby požadavky nepropadaly řetězem k terminálnímu objektu.
- **Příležitosti** – Komplexní rozhodovací logika, která řídí tok programu, může být rozložena na posloupnost jednodušších rozhodovacích kroků.
- **Hrozby** – Při minimálním počtu větvení a minimálním podmíněném chování se stává *Chain of Responsibility* zbytečnou zátěží. Je potřeba dobře vybalancovat přínos a čas návrhového vzoru.

Strategy

Návrhový vzor *Strategy* řeší nešváry ze třídy vazbičů. Jeho cílem je umožnit volbu strategií, které se využijí k řešení daného problému, bez závislosti na konkrétní implementaci. Pokud například vezmeme problém řazení prvků, pak řadící algoritmy mohou být reprezentovány jako strategie, a můžeme je za běhu programu volit podle potřeby.



Obrázek 2.13: Komutativní diagram návrhového vzoru: Strategy

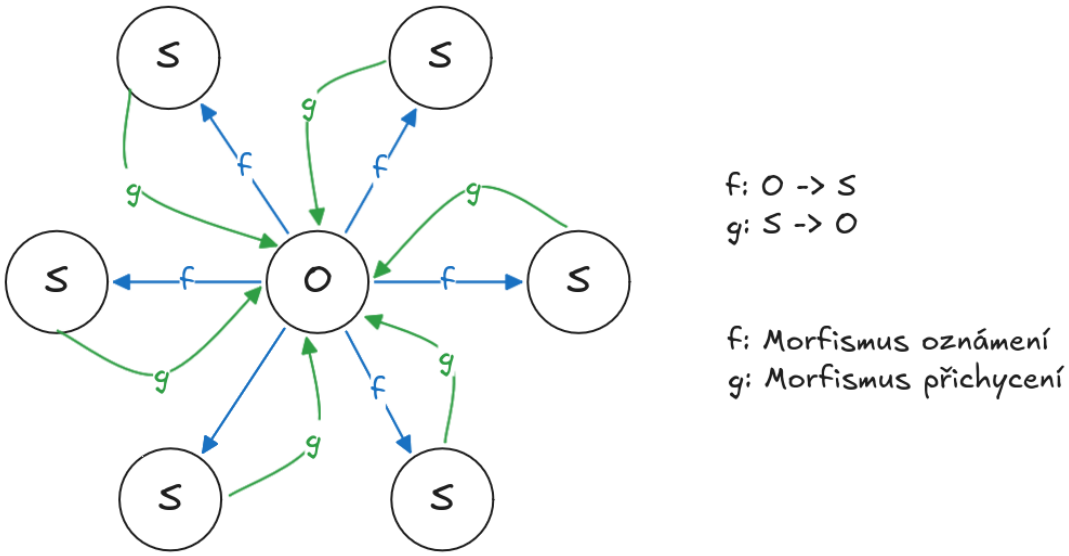
Podíváme-li se na komutativní diagram 2.13, je zřejmé, že vypadá velmi podobně komutativnímu diagramu 2.8 návrhového vzoru *Factory*. Není tomu tak náhodou. Zatímco návrhový vzor *Factory* abstrahuje a deleguje vytváření objektů, návrhový vzor *Strategy* abstrahuje a deleguje vykonávání metod. Návrhový vzor *Factory* vede k instanci objektu, *Strategy* vede k výsledku operace.

Na základě výše uvedené formální definice lze provést systematické vyhodnocení přínosů a rizik návrhové vzoru *Strategy*:

- **Silné stránky** – Složitá, externí či nízkoúrovňová rozhraní, které nelze refaktorizovat, mohou být extrahovány do sémanticky smysluplných rozhraní.
- **Slabé stránky** – Závislost dalších komponent na singletonu může zavést vedlejší účinky do dalších objektů.
- **Příležitosti** – Zjednodušení rozhraní externích systémů a zapouzdření běžných operací v rámci domény.
- **Hrozby** – Předčasné zapouzdření do návrhového vzoru *Facade* místo refaktorizace, když je to možné. Nadbytečné užívání návrhového vzoru tam, kde to nemá z hlediska zlepšení sémantiky význam.

Observer

Návrhový vzor *Observer* řeší nešváry ze třídy vazbičů a zabraňovačů. Řeší je tak, že vytváří událostmi řízený mechanismus pro odesílání a přijímání zpráv.



Obrázek 2.14: Komutativní diagram návrhového vzoru: Observer

Formálně je návrhový vzor *Observer* kategorií o dvou objektech (*Observer a Subscriber*) a dvou morfismech, které reprezentují životní cyklus událostí. Objekt *Observer* vystupuje jako centrální uzel, který pomocí morfismu oznámení iniciuje odeslání zprávy. Objekty *Subscriber* se přihlásí k přijímání zpráv pomocí morfismu přichycení. Komutativita diagramu 2.14 je zajištěna právě díky tomu, že každý *Subscriber* je ve smyčce s objektem *Observer*. To je důležité proto, že není žádoucí, aby *Observer* posílal zprávy objektům *Subscriber*, které se k jejich přijímání nepřihlásili, a naopak, pokud se přihlásí pomocí morfismu přichycení, musí být o nových zprávách informováni.

Na základě výše uvedené formální definice lze provést systematické vyhodnocení přínosů a rizik návrhové vzoru *Observer*:

- **Silné stránky** – Dovoluje libovolnému počtu objektů sledovat události, které vyvolávají morfismus oznámení. Tyto objekty nemusí být na sobě přitom nijak závislé.
- **Slabé stránky** – Po oznámení události se ztrácí vazba na místo volání.
- **Příležitosti** – Pokud je potřeba do programu zavést událostmi řízené programování, například volání více izolovaných funkcí z jednoho místa.
- **Hrozby** – Může být spuštěna kaskáda událostí, která může zahltit systém a udělat ho netrasovatelným.

2.3.4 Kompozitní vzory

Návrhové vzory popsané v předchozí kapitole nepředstavují izolované ani disjunktní konstrukce. Jak je zmíněno již na začátku kapitoly, návrhové vzory se přizpůsobují specifikům nejen jazyka, ale i konkrétního systému, což přirozeně vede k jejich vzájemnému prolínání. Těmto zkombinovaným vzorům říkám *kompozitní vzory*.

Kompozitní vzory nám umožňují integrovat typicky dva strukturálně kompatibilní diagramy do jedné kategorie, přičemž nesmí dojít ke konfliktu v rozdělení odpovědností. V předchozích definicích jsem se záměrně soustředil na vnitřní topologii a vnitřní morfismy jednotlivých vzorů namísto fixace jejich vstupních a výstupních rozhraní. Tento přístup nám nyní dovoluje pohlížet na vzory jako na modulární komponenty grafu objektů. Můžeme se tak plně soustředit na syntézu jejich funkcionality a zkoumat, jak se vnitřní struktura jednoho vzoru stává přirozeným stavebním kamenem vzoru druhého.

Pozastavme se nad podmínkou nenarušení rozdělení odpovědností mezi návrhovými vzory. Tuto podmínku vkládám i z formálního hlediska, kdy vymezuje hranice strukturální podobnosti mezi jednotlivými vzory. Abychom mohli o kompozici uvažovat jako o validní operaci nad kategoriemi, musí být zachována sémantická ekvivalence vnitřních procesů každého zúčastněného vzoru. Pokud objekt v diagramu kumuluje protichůdné odpovědnosti, dochází k rozpadu izomorfismu s ideálním modelem.

Případ, kdy taková kompozice funguje, je složení návrhového vzoru *Factory* a *Strategy*. Jak jsem již poukázal u kumulativního diagramu 2.13, tyto dva vzory sdílejí z velké části kompatibilní vnitřní topologii. V případě továrny strategií je produkt P_n sémanticky ekvivalentní se strategií S_n, protože ta konkrétní strategie je právě tím produktem, který má vzor *Factory* vytvořit. Když tedy komutativní diagramy těchto dvou vzorů prostě zřetězíme, zjistíme, že vzniká složený morfismus x. Protože jsou produkty a strategie sémanticky ekvivalentní $S_N \approx P_N$, můžeme diagram redukovat, přičemž musí být zachovány morfismy tvorby, abstrakce a kontraktu ze strany vzoru *Factory* a morfismy kontraktu a exekuce ze strany vzoru *Strategy*.

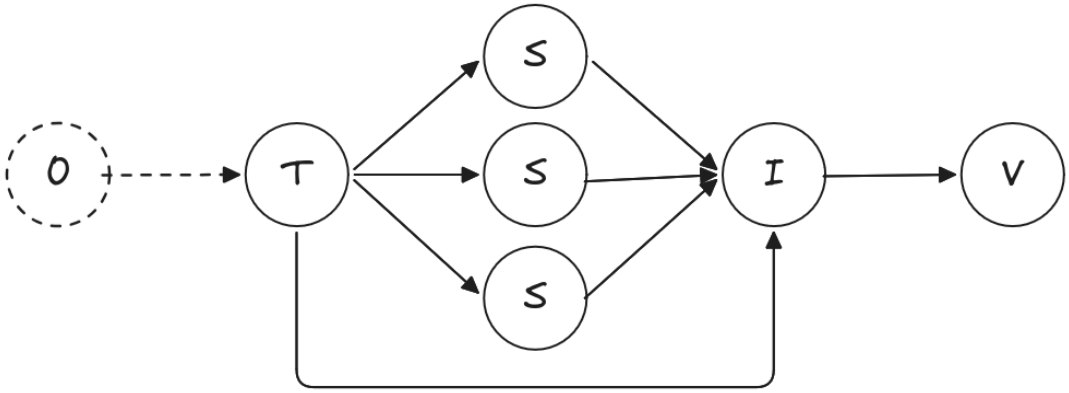
$$x = 0 \to T \to P_n \to I \to I \to S_n \to V \tag{2.12}$$

$$x = 0 \to T \to S_n \to I \to I \to V \tag{2.13}$$

$$x = 0 \to T \to S_n \to I \to V \tag{2.14}$$

Při této syntéze slouží rozhraní I jako společný uzel, kde se oba vzory potkávají. Vzor *Factory* má za úkol objekt vyrobit a předat ho právě tomuto rozhraní. V tu chvíli přebírá štafetu vzor *Strategy*, který rozhraní využije k tomu, aby provedl výpočet a dodal výsledek. Vzniká tak plynulá cesta od výroby až k cíli. Rozhraní I v grafu musí zůstat, protože funguje jako

pojistka: zaručuje, že strategie dostane přesně takový nástroj, jaký ke své práci potřebuje.



Obrázek 2.15: Kompozitní návrhový vzor: Factory of Strategies

Existuje ale i případy, kdy je syntéza návrhových vzorů logicky neproveditelná. K selhání dochází v momentě, kdy se střetnou základní strukturní pravidla. Pokud jeden vzor vyžaduje uspořádání, které přímo popírá princip vzoru druhého, vzniká neřešitelný rozpor. V takovém případě uzel v komutativním diagramu nedokáže plnit obě role současně, aniž by nedošlo k narušení stability celého systému.

Příkladem strukturního konfliktu je pokus o sloučení vzorů *Chain of Responsibility* a *Singleton*. Zatímco *COR* je z definice založen na posloupnosti několika různých objektů, které si předávají požadavek, *Singleton* striktně vynucuje existenci pouze jediné instance. Pokud bychom tyto vzory chtěli složit do jednoho vzoru, řetězec by se strukturálně rozpadl a efektivně utvořil návrhový vzor *Facade*.

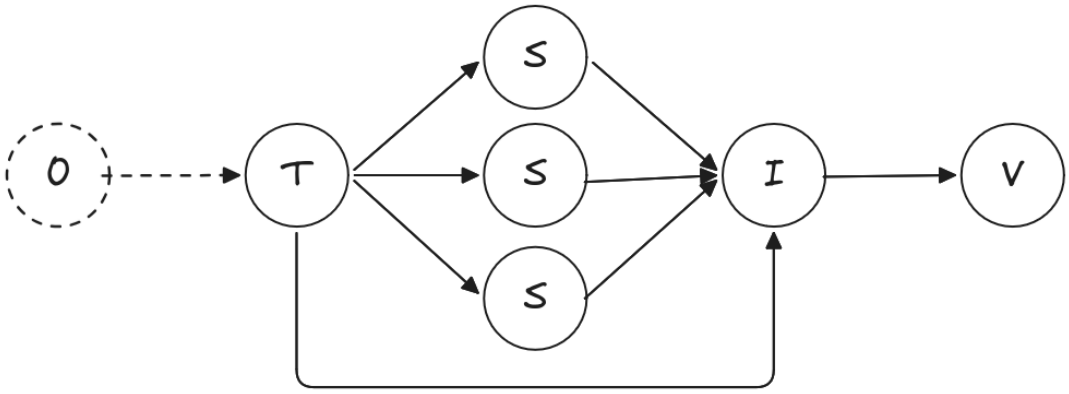
$$COR = C \to H_1 \to H_2 \to H_3 \to \cdots \to H_n \to C \qquad (H_i \neq H_j) \tag{2.15}$$

$$x = C \to S_1 \to S_2 \to S_3 \to \cdots \to S_n \to C \qquad (H_n \approx S_n) \tag{2.16}$$

$$x = C \to S \to C \qquad (S_1 = S_2 = \cdots = S_n) \tag{2.17}$$

$$x \not\sim COR \tag{2.18}$$

Morfismus předání by v tomto bodě přestal existovat, protože by objekt neměl na koho jiného delegovat než sám na sebe, čímž se celá struktura zhroutí.



Obrázek 2.16: Kompozitní návrhový vzor: Factory of Strategies

Další bariérou pro úspěšnou kompozici je konflikt v životním cyklu objektů, který nastává například při spojení vzorů *Builder* a *Strategy*. *Builder* je navržen pro postupné sestavování složitého objektu, který je po celou dobu procesu nekompletní. Naproti tomu Strategy vyžaduje plně inicializovaný a stabilní objekt, aby mohla okamžitě vykonat svůj algoritmus. Odpovědnost za tvorbu a odpovědnost za výkon se zde časově vylučují; pokud bychom nutili nehotový *Builder* vykonávat strategii, výsledek by byl nepředvídatelný.

Princip jedné odpovědnosti není tedy jen otázkou čistoty kódu, ale i formální správnosti. Jeho porušení vede k tomu, že výsledný diagram nelze logicky uzavřít – chybí mu jasný směr toku informací nebo terminální uzel s výsledkem. Úspěšná kompozice je tedy možná pouze tehdy, pokud spolu vzory spolupracují v čase i prostoru diagramu, jako je tomu u výše zmíněné kombinace vzorů *Factory* a *Strategy*.

2.3.5 Závěr

V této kapitole jsem nastínil problematiku návrhových vzorů a snažil jsem se o její formalizaci pomocí teorie kategorií. Je nutné ovšem podotknout, že plná formalizace není v mé kompetenci. Využil jsem pro mě známý nástroj, ale v rámci použitelnosti a přístupnosti jsem vynechal spoustu rigoru, který by si tato tématika zasloužila.

Je totiž dost možné, že při hlubším zkoumání mého postupu budou mé myšlenky o teorii kategorií jakožto formálním základu návrhových vyvráceny, či vehementně opraveny. V současné době je to ale pro mě dostatečně silný analytický nástroj, kterým mohu alespoň kvazifomálně popsat strukturu kódu a porovnávat ho s návrhovými vzory pomocí relací.

2.4 ARCHITEKTONICKÉ VZORY

Jestliže návrhové vzory představují osvědčené strategie pro uspořádání vnitřních struktur jednotlivých komponent, architektonické vzory posouvají tuto abstrakci na další úroveň. Zatímco v předchozí kapitole jsme se soustředili na vztahy mezi objekty, zde se zaměřujeme na systém jako celek. Architektonické vzory definují globální organizační schéma a určují základní pravidla pro rozdělení odpovědností mezi rozsáhlé podsystémy, moduly či vrstvy (GeeksForGeeks, 2025).

V první řadě musím vymezit úroveň abstrakce, na které se v této kapitole pohybuji. Často se totiž lze setkat s pojmy jako *architektura klient-server* nebo *monolitická architektura*, přičemž jsou chybně stavěny na stejnou úroveň jen proto, že sdílejí společné označení. Je však zásadní rozlišovat architekturou zaměřenou na vnitřní uspořádání aplikace a architekturou řešící interakci mezi systémy. Zatímco monolitická architektura je vzor určující vnitřní strukturu programu, model klient-server zase definuje styl propojení v rámci distribuovaného systému. V této kapitole se tedy zaměřuji výhradně na vzory uvnitř hranic jednoho softwarového řešení, zatímco interakci mezi více programovými celky řeším v kapitole 2.5.

2.4.1 Koncept softwarové architektury

Každé programové řešení, bez ohledu na jeho rozsah, je podrobeno určité architektuře, která se formuje jako úhrn návrhových rozhodnutí v čase. Architekturu v tomto smyslu nelze vnímat jako statický plán, nýbrž jako dynamický proces neustálého přidávání, odebírání a modifikací prvků systému (Jansen; Bosch, 2005). Každé takové rozhodnutí představuje cílený zásah do struktury řešení a je definováno svým specifickým kontextem.

Tento kontext tvoří především architektonické zdůvodnění, které osvětluje příčiny volby daného postupu. Rozhodovací proces je dále vázán návrhovými pravidly a omezeními, jež vymezují prostor přípustných řešení a zajišťují vnitřní integritu systému. V neposlední řadě jsou tato rozhodnutí přímou odpovědí na stanovené požadavky. Architektura tedy není pouze výsledným stavem komponent, ale je tvořena celou historií těchto rozhodnutí, z nichž každé nese informaci o své motivaci, pravidlech a limitech, za kterých dané rozhodnutí vzniklo (Jansen; Bosch, 2005).

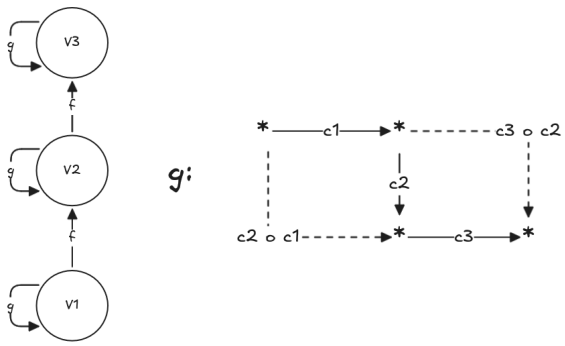
Opakující se povaha architektonických rozhodnutí vede k pozorování, že mnohé problémy v rozdělení odpovědnosti a v uspořádání komponent vykazují strukturní podobnost. Podobně jako se na úrovni implementace jednotlivých komponent vyvinuly návrhové vzory pro řešení lokálních problémů, v širším kontextu uspořádání celého systému postupem času vykrystalizovaly architektonické vzory. Ty nepředstavují pouze hotové šablony, ale spíše formalizované sady architektonických rozhodnutí, které v minulosti úspěšně vyřešily specifické kombinace požadavků a omezení.

2.4.2 Popis vybraných architektonických vzorů

Zatímco v kapitole věnující se návrhovým vzorům byla teorie kategorií využita jako prostředek pro jejich formální popis, v této části popíšu pomocí stejné metodologie úroveň architektonických vzorů. Aplikace tohoto formalismu na makroarchitektonické úrovni poskytuje objektivní rámec pro posouzení strukturní integrity celého systému, neboť nám dovoluje identifikovat porušení definovaných hierarchických pravidel — například v podobě nechtěných cyklů nebo obcházení vrstev — jako exaktně popsané nekomutativní prvky, které v implementační praxi představují úmyslný či neúmyslný technický dluh.

Vrstvená architektura

Vrstvená architektura rozkládá programové řešení do sady úrovní (vrstev), které mají rozdělené konkrétní odpovědnosti a jednotnou úroveň abstrakce. (Khan, 2023)



Obrázek 2.17: Komutativní diagram vrstvené architektury

Jak je zakresleno v diagramu 2.17, vrstvy jsou uspořádány ve vertikální hierarchii. Samotné rozdělení do vrstev se používá k definici úrovní abstrakce: každá vrstva poskytuje rozhraní vrstvě bezprostředně nadřazené. Tato struktura umožňuje na nejvyšší úrovni skládat komplexní doménové operace s využitím nízkoúrovňových nástrojů, které jsou zapouzdřeny v nižších vrstvách. Významným přínosem tohoto uspořádání je pak vysoká míra modularity, díky níž lze jednotlivé vrstvy vyvíjet, optimalizovat a testovat v izolaci. (Khan, 2023)

Z formálního hlediska klademe na vrstvenou architekturu $A = (\mathcal{O}, \mathcal{H})$ několik striktních omezení a předpokladů. Jednotlivé vrstvy systému jsou reprezentovány jako monoidy, které v sobě zapouzdřují vnitřní logiku dané úrovně. Ke každému monoidu vrstvy náleží identický endomorfismus $g : V \to V$ a homomorfismus abstrakce $f_{i,j} : V_i \to V_j$, pro který platí podmínka návaznosti $j = i + 1$. Z těchto definic odvozujeme následující strukturální vlastnosti:

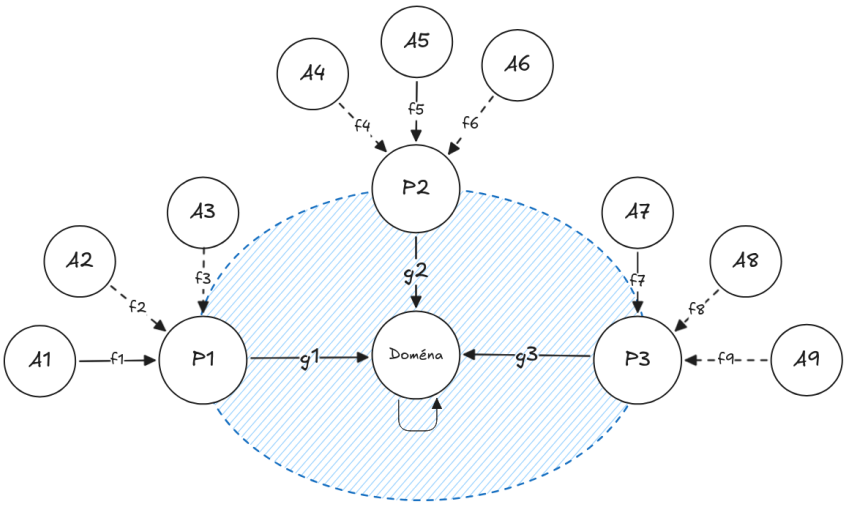
- **Vnitřní integrita** - Každá vrstva zapouzdřuje své komponenty ve formě vnitřních endomorfismů. Využití komponent v rámci stejné vrstvy je výsledek skládání vnitřních morfismů, které nemění výsledný objekt (zůstáváme pořád ve stejné vrstvě).
- **Omezení vertikálního prostupu** - Každá vrstva smí využívat pouze komponenty své a následné vrstvy. Toto omezení plyne z pravidla následnosti a existuje proto, abychom eliminovali nekontrolované skoky v hiearchii.
- **Omezení zpětného prostupu** - Nižší vrstva nemůže využívat komponenty vyšší vrstvy. Každá vrstva určuje nejvyšší úroveň abstrakce. Pokud bychom tuto podmínku porušili, budou mezi vrstvami rozmazány hranice abstrakce a jednotlivé vrstvy se defacto zhroutí do jedné.

Vrstvená architektura primárně řeší problémy spojené s rozdělením úrovní abstrakce, což umožňuje členům vývojového týmu soustředit se na svou specifickou expertízu, aniž by museli detailně rozumět implementaci zbytku aplikace. Tyto pozitivní vlastnosti jsou však přímo podmíněny striktním dodržováním strukturálních pravidel, což s sebou přirozeně nese i určitá negativa. Při vyšším počtu vrstev totiž dochází k nárůstu režie, kdy je nutné i jednoduché operace propisovat skrze celou hierarchii, což může vést k nadbytečné tvorbě kódu, který data pouze pasivně předává. Navíc každá vrstva zůstává funkčně úzce spjata s vrstvou bezprostředně pod ní, takže i když jsou od sebe logicky izolovány, jakákoliv zásadní změna v základech systému se může kaskádovitě promítnout do všech nadřazených pater. (Khan, 2023)

Hexagonální architektura

Hexagonální architektura představuje alternativu k vrstvené architektuře. Zatímco vrstvy se soustředí na hierarchii, tento vzor se zaměřuje na izolaci doménové logiky od vnějších technologií. Cílem je, aby jádro aplikace bylo nezávislé na tom, zda komunikuje s uživatelem přes webový prohlížeč, mobilní aplikaci nebo zda ukládá data do SQL či NoSQL databáze. (Cockburn, 2005)

Z hlediska formálního popisu definujeme hexagonální architekturu jako $A = (\mathcal{O}, \mathcal{H})$, jejíž centrálním objektem je doménové jádro D. Toto jádro zapouzdřuje čistou byznys logiku, nezávislou na okolních technologiích. Na hranici domény definujeme porty P, které tvoří abstraktní rozhraní systému. K těmto portům přistupují adaptéry A, které jsou konkrétními implementacemi k rozhraní, které je portem definováno.



Obrázek 2.18: Komutativní diagram hexagonální architektury

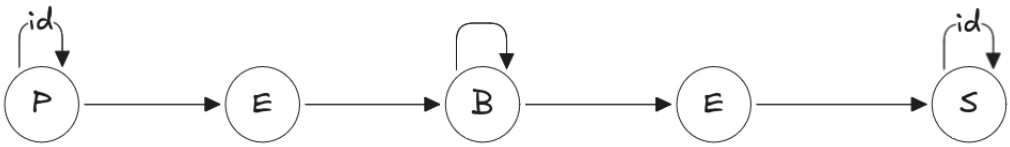
Vzhledem k navrženému diagramu 2.18 lze na výsledný běžící systém pohlížet jako na výsledek strukturální konvergence. Na počátku životního cyklu systému existuje architektura jako abstraktní kategorie s mnoha stupni volnosti, což reprezentujeme třídami sémanticky ekvivalentních morfismů (čárkované čáry v diagramu). Proces konvergence pak probíhá následovně:

- **Redukce stupňů volnosti** – Ve fázi inicializace systému dochází k výběru konkrétních adaptérů pro každý definovaný port. Tím se původně variabilní diagram ustálí do pevné topologie, kde jsou teoretické možnosti nahrazeny aktivními vazbami.
- **Pohlcení adaptérů do portů** – Skrze operaci skládání systém konverguje do stabilního stavu. V tomto bodě dochází k uplatnění izomorfismu mezi aktivním adaptérem a příslušným portem: $A \cong P \wedge \exists f : A \to P$.
- **Pohlcení portů do domény** – Jelikož jsou porty z principu integrální součástí domény, závěrečná fáze konvergence představuje jejich transformaci na aktivní vnitřní morfismy monoidu D. Původně pasivní rozhraní jsou absorbována do vnitřní logiky jádra.

Hexagonální architektura přeměňuje rizika vrstvené architektury v konkureční výhodu. Ve vrstvené architektuře je totiž zcela možné, aby do vyšších pater prosákly i artefakty, které nejsou součástí kontraktu s další vrstvou. Hexagonální model však tento jev mění v řízenou strukturální konvergenci. Namísto vrstev používá porty jako striktní filtry, které nepřipouští vznik morfismů mimo kontrakt. Systém se tak úmyslně nechává zhroutit do jediného stabilního bodu, kde se adaptéry a porty vzájemně pohltí. Tím se celá architektura bezpečně uzamkne do vnitřně konzistentního monoidu domény.

Událostmi řízená architektura

Událostmi řízená architektura přenáší principy návrhového vzoru Observer (kapitola 2.3.3) na úroveň celého systémového řešení. Komunikace mezi komponentami v tomto modelu neprobíhá skrze přímé synchronní volání, ale prostřednictvím zpráv (událostí), které jsou distribuovány mezi zúčastněné celky skrze nezávislé prostředníky.



Obrázek 2.19: Komutativní diagram událostmi řízené architektury

Událostmi řízená architektura mění způsob komunikace mezi komponentami ze sychronní komunikace, ve které figuruje požadavek a odpověď, na asynchronní model zpracování událostí. Události jsou zde prvořadí nositelé stavu, na které může reagovat nespočetné množství na sobě nezávislých komponent (Pallaprolu, 2025). V událostmi řízené architektuře v důsledku nízkého provázání vzniká několik problémů, na které je nutné si dávat pozor. Asynchronní podstata zpracování události vnáší následující problémy (Pallaprolu, 2025):

- **Nekonzistence systému** – Při zpracování události několika komponentami nemůžeme zaručit, že všechny komponenty zpracují událost ve stejném čase, tudíž může být systém v čase nekonzistentní s nejasnou dobou ustálení.
- **Pořadí zpracování událostí** – Při zpracování události není jasné, v jakém pořadí budou komponenty událost zpracovávat. Pokud tak jedna komponenta závisí na druhé, vzniká logický rozpor.
- **Problémy s bezpečností** – Komponenty nemají tušení, kdo událost vyvolal. Je nutné navrhnout bezpečností model pro autentizaci a autorizaci uživatelů tak, aby nespoléhal na implicitní zabezpečení pomocí vazeb mezi komponentami. Každá komponenta si musí bezpečnost ověřit samostatně.

Událostmi řízená architektura představuje efektivní řešení problému úzkého provázání komponent, ovšem za cenu ztráty temporální konzistence systému. Toto pozorování v jádru odpovídá CAP teorému z oblasti distribuovaných systémů. V okamžiku, kdy rozbijeme synchronní komunikaci na události s asynchronním zpracováním, vědomě se vzdáváme okamžité konzistence ve prospěch odolnosti vůči rozdělení sítě a dostupnosti.

Dostupnost se v tomto modelu projevuje schopností systému přijímat vstupy bez ohledu na aktuální stav navazujících komponent, neboť prostředník fungující jako vyrovnávací paměť eliminuje kaskádová selhání a dovoluje producentovi úspěšně odeslat událost i při přetížení či dočasném odpojení cílové domény. Odolnost vůči rozdělení sítě pak v praxi zajišťuje, že přerušení komunikačního kanálu nezpůsobí kolaps systému; prostředník zprávu uchová a doručí ji až po obnovení integrity sítě.

Tato provozní kontinuita je však vykoupena zánikem atomicity operací a přechodem k režimu konečné konzistence, což s sebou přináší i specifické riziko sémantické oscilace. K té dochází v momentě, kdy zřetězené události vytvoří v diagramu uzavřenou smyčku, v níž jedna doména zpětně vyvolává reakci v doméně předchozí, čímž systém namísto konvergence k rovnovážnému stavu generuje nekonečnou řadu divergentních operací.

2.4.3 Závěr

V této části jsem popsal tři vzory, které řeší provázání prvků v systému. Vybral jsem je záměrně, aby ukázaly jasný trend: postupné uvolňování vazeb mezi doménami. Začal jsem u vrstev, kde jsou spoje pevné a přímé. Pokračoval jsem k hexagonu, který vazby izoluje do portů a chrání tak jádro. Celý trend vrcholí v asynchronním modelu událostmi řízené architektury, kde se vazba zcela rozpadá. Zde již prvky o své existenci neví a všechnu komunikaci řeší prostředník. Každý krok sice zvyšuje svobodu a izolaci kódu, ale zároveň ztěžuje sledování toku dat a zvyšuje riziko časové nekonzistence. S rostoucí volností tedy roste i režie na správu celého systému.

Podobně jako u návrhových vzorů, i zde lze provést složení do složitějších celků. Tyto vzory se totiž nevylučují. Do jádra hexagonu mohu vložit vrstvy pro vnitřní řád, nebo celou doménu zapojit jako jeden bod do sítě řízené událostmi. Tato kompozice mi dovoluje volit míru izolace přesně podle potřeb dané části systému.

2.5 INTEGRAČNÍ VZORY

Přesouváme se do poslední úrovně uspořádání programového řešení, kdy nyní již nebudeme řešit vnitřní strukturou jednotlivých komponent, ale zaměřuji se na uspořádání programových celků jako takových. Integrační vzory představují pravidla pro orchestraci a skládání autonomních služeb do uceleného systému. Zatímco v předchozích částech bylo řešeno, jak kód izolovat uvnitř jednoho celku, zde je zaměření na systém, jakým tyto celky definovat navenek a jakými sémantickými kanály je propojit.

2.5.1 Popis vybraných integračních vzorů

Podobně jako v případě návrhových a architektonických vzorů jsem i zde vybral zástupce, kteří stojí při volbě vzoru za úvahu. Na rozdíl od předchozích kapitol se však v této části zdržím formálního popisu pomocí teorie kategorií. K tomuto kroku jsem přistoupil proto, že v matematické formalizaci integračních vzorů neshledávám takový přínos pro analýzu technického dluhu jako u nižších úrovní abstrakce. Zatímco u vnitřní struktury komponent je přesné vymezení morfismů zásadní pro detekci nelegitimních vazeb, u integračních vzorů je podstata problému spíše v sémantickém nastavení hranic a v distribuované povaze systému.

Monolitická architektura

Monolitická architektura představuje pravděpodobně nejčastější model integrace. Celé řešení je sestaveno jako jeden nerozdělitelný celek. V rámci tohoto uspořádání spolu jednotlivé komponenty komunikují pomocí přímého volání funkcí či metod v operační paměti. K předávání dat a řízení mezi různými částmi systému se využívají přímé reference, ukazatele na funkce či sdílený stav (Khan, 2023).

Hlavní předností monolitu je soustředění kódu do jednoho celku. Tato blízkost dovoluje přímý kontakt s daty či úložištěm bez nutnosti "hopsat" přes řetězec dalších služeb. Díky tomu klesá latence, kterou v síti vyvolává režie spojení a komunikace mezi uzly. Monolit tak nabízí nejnižší možné zpoždění, neboť se na externí zdroje napojuje přímo.

Nevýhodou monolitu je však horší možnosti škálování. Pokud je nápor vyvíjen pouze na jeden modul, musíme navýšit výkon pro celý systém. Z pohledu technického dluhu jde o rizikové prostředí, protože fyzická blízkost prvků totiž svádí k tvorbě vazeb, které nedávají smysl. Absence jasných hranic klade nároky na kázeň vývojářů. Každý chybný krok či špatné vytvoření vazby se v monolitu šíří snáze než tam, kde existují pevné hranice.

Mikroslužbová architektura

Právě omezená škálovatelnost vedla ke vzniku mikroslužeb. Tento přístup dělí systém do sady malých, nezávislých prvků. Každá služba běží ve vlastním procesu a s okolím komunikuje přes jasné rozhraní, čímž izoluje svou vnitřní implementaci (James Lewis, 2014). Hlavní výhodou je pro mě možnost škálovat pouze ty části systému, které jsou pod tlakem.

Tato výhoda je ale podmíněna tím, že je systém rozdělen na mikroslužby správně. Pokud hranice služeb neodpovídají logickým doménám, mohou vznikat megaslužby – tedy monolitické celky, které pouze využívají distribuované prostředí, ale vnitřně zůstávají neúměrně složité. V takovém stavu jsou služby sice fyzicky oddělené, ale sémanticky natolik provázané, že jakákoliv změna v jedné části vynucuje úpravy a nasazení všech ostatních uzlů najednou.

Klient-Server architektura

Architektura klient-server nahlíží na dělení systému skrze sémantiku rolí. Programové celky dělí na klienty, kteří tvoří požadavky, a servery, které na ně odpovídají (Oluwatosin, 2014). Velkou výhodou je, že k jedné sadě zdrojů na serveru může přistupovat více různých klientů najednou. To dovoluje sdílet data i logiku napříč různými rozhraními.

Toto jasné rozdělení rolí sice zjednodušuje správu dat a řízení přístupu, ale zároveň vytváří silnou závislost klienta na dostupnosti serveru. Z pohledu vazeb jde o vertikální vztah, kde je stabilita celku přímo úměrná stavu centrálního uzlu. Pokud není dostupný žádný server, klienti se stávají nepoužitelnými.

2.6 ZÁVĚRY

V této kapitole jsem rozebral technický dluh a jeho projevy na různých úrovních abstrakce. Mým cílem bylo ukázat, že dluh není homogenní entita, ale nabývá specifických forem podle toho, na jaké úrovni abstrakce se nacházíme. Pro každou z těchto vrstev jsem zvolil takové zástupce a vzory, které nejlépe řeší rizika vzniku technického dluhu.

Tam, kde to bylo z pohledu analýzy přínosné, jsem tyto vzory formalizoval pomocí teorie kategorií. To mi dovolilo vytvořit formální popisy vzorů, které lze následně porovnávat s reálnými ukázkami v produkčním kódu. U vyšších úrovní integrace jsem se naopak soustředil na slovní a logický rozbor, neboť zde podstata problému leží spíše v distribuci odpovědnosti než v samotné vnitřní struktuře.

Je důležité podotknout, že pro každou úroveň existuje celá řada dalších vzorů a způsobů, jak technický dluh řešit. Můj výčet tedy není taxativní. Představuje však dostatečný základ pro pochopení dané problematiky v kontextu této práce a jak s těmito vzory budu pracovat dále. Tabulka níže porovnává jednotlivé úrovně abstrakce s principy pro řízení technického dluhu, respektive kvality kódu.

Úroveň C4	Název úrovně	Princip řízení kvality kódu	Kapitola
Úroveň 4	Systémový kontext	Integrační vzory	2.5
Úroveň 3	Kontejnery	Architektonické vzory	2.4
Úroveň 2	Komponenty	Návrhové vzory	2.3
Úroveň 1	Kód	Čistý kód	2.2

Tabulka 2.2: Propojení úrovní modelu C4 s principy softwarového inženýrství

3 UDRŽITELNOST PRODUKČNÍHO KÓDU

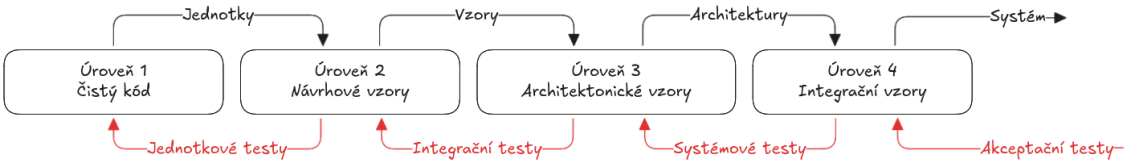
Předchozí kapitola se věnovala principům softwarového inženýrství, které brání vzniku technického dluhu skrze vnitřní uspořádání systému. Tyto principy vycházely především z intrinsivních charakteristik kódu – tedy z toho, jak je kód strukturován, jak jsou v něm definovány vazby a jakým způsobem jsou domény izolovanány uvnitř vývojového prostředí. Stejně důležitou součástí životního cyklu je však zkoumání a dokumentování extrinsivního chování, tedy toho, jak se systém projevuje navenek v reálném provozu.

V této kapitole obracím pozornost k fázi, která nastává poté, co se dopíše poslední řádek kódu. Začíná tedy uvolněním kandidátní verze a pokračuje až po ostré nasazení do produkce. Jako čistý kód či návrhové vzory komunikují záměr na úrovni úrovní abstrakce kódu, nyní se soustředíme na ověřování těchto záměrů a zda jim chování systému odpovídá.

3.1 TESTOVÁNÍ

První fází analýzy vnějšího chování je testování. To neslouží k prostému ověření, že systém funguje, ale k nalezení co největšího počtu chyb. Testy se dělí na dvě základní strategie podle přístupu k informacím o kódu. Black-box testování nahlíží na program jako na uzavřený celek a zkoumá pouze odezvu na různé kombinace vstupů. Naproti tomu White-box testování využívá znalost vnitřní logiky jednotky, na jejímž základě pak cíleně vytváří testovací případy (Myers et al., 2004).

Pokud chceme testováním efektivně bránit vzniku technického dluhu, musíme prověřovat stejné úrovně, na kterých se tento dluh vyskytuje. Jelikož úrovně abstrakce definované v kapitole 2.1 tvoří soustavu vrstev, lze zde aplikovat myšlenky z vrstvené architektury. Každá nižší vrstva poskytuje rozhraní vrstvě vyšší, která její zdroje využívá k syntéze nové funkcionality. Testování by mělo tento princip napodobovat, aby věrně simulovalo chování systému i budoucích vývojářů. Konkrétní úroveň abstrakce je proto ideální testovat z pohledu té vyšší, která k ní přistupuje jako ke službě. Tímto způsobem nekontrolujeme pouze izolovanou logiku, ale především integritu a stabilitu kontraktů mezi celky.



Obrázek 3.1: Úrovně abstrakce a typy testů

Je však nutné rozlišit mezi dluhem samotným a jeho projevy. Testování zřídka odhalí technický dluh přímo v jeho vnitřní formě; málokdy nám test řekne, že kód je špatně strukturovaný nebo že porušuje principy Clean Code. Od toho je statická analýza kódu a diskuze. Testování však odhaluje dopady tohoto dluhu na vnější chování systému. Pokud se v důsledku vnitřní degradace začne rozpadat konzistence dat nebo selhávají dříve funkční rozhraní, jsou to právě testy, které na tyto symptomy upozorní. Testování tak slouží jako indikátor zdraví celku. Ačkoliv neodhalí špatný návrh v momentě jeho vzniku, okamžitě zachytí jeho následky.

Přínosy testování jsou však přímo závislé na kvalitě zvolených testovacích případů. Již z principu nelze otestovat všechny možné vstupní hodnoty, například celý rozsah celých čísel, protože by takový proces byl časově i výpočetně neúnosný. Proto se využívá metoda rozdělení na třídy ekvivalence. Tato technika spočívá v rozčlenění vstupních dat do skupin, u nichž se předpokládá, že je systém zpracuje sémanticky stejným způsobem. Z každé takové třídy se následně vybere reprezentativní vzorek, jehož výsledek zastupuje chování celého bloku hodnot (Copeland, 2004). Pokud test uspěje pro jednoho zástupce, považujeme celou třídu za bezchybnou, čímž radikálně snižujeme počet nutných testů při zachování vysoké míry důvěry v systém.

U ordinálních proměnných, tedy hodnot, které mají definované pořadí, je však nutné tento přístup doplnit o metodu okrajových hodnot. Statistické zkušenosti ukazují, že technický dluh se nejčastěji projevuje právě na hranicích definovaných tříd ekvivalence – v místech, kde se mění logika zpracování nebo kde dochází k přetečení datových typů. Místo náhodného vzorku z vnitřku třídy se proto zaměřujeme na její samotné okraje a hodnoty těsně za nimi. Pokud tyto hodnoty vyvolají správnou reakci, získáváme jistotu, že vnitřní implementace věrně odpovídá navrženému modelu.

Problematika testování je rozsáhlá disciplína, kterou nemá tato práce šance ani zdaleka pokrýt. Pro účely této práce je však klíčové především pochopení vazby mezi druhy testů a jednotlivými úrovněmi abstrakce. Protože nelze testovat všechny možné kombinace, musí být zvolena vhodná strategie pro výběr testovacích dat. K tomuto účelu plně postačuje dříve popsané rozdělení vstupů do tříd ekvivalence a následné zacílení na jejich reprezentativní vzorky a okrajové hodnoty.

3.2 OBSERVABILITA

Ačkoliv je primárním cílem testování odhalení chyb v programovém řešení, z principu nelze zaručit zachycení všech defektů před samotným nasazením. Pokud se chyba projeví během vývoje nebo v testování, má vývojář k dispozici luxus v podobě debuggeru. Ten mu dovoluje odkrokovat program a sledovat vnitřní stav systému v reálném čase.

V produkčním nasazení však debugger zpravidla k dispozici nemáme. Další překážkou jsou chybějící pomocné struktury, jako jsou tabulky symbolů, nebo agresivní optimalizace kompilátoru. V momentě, kdy není možné sledovat stav aplikace zevnitř, nezbývá než chování sledovat zvenčí. Schopnost pozorovat chování systému na základě jeho výstupů nazývám observabilitou.

Mnoho autorů a spolků, kteří se observabilitou zabývají, jí rozdělují na pilíře. Každý z těchto pílířů odpovídá na jednu z následujících otázek: *Co se to děje?*, *Proč se to děje?* a *Kde se to děje?*. Každý pilíř tak popisuje chování systému z jiného pohledu a dohromady tvoří kompletní informaci. Mezi tyto pilíře patří logy, metriky a trasy (Elastic Observability Team, 2024). Tato práce však na observabilitu nahlíží jinak. Tradiční paradigma pilířů totiž přirozeně evokuje představu, že se jedná o samostatné články, jejichž spojením vzniká celková observabilita.

Vzhledem k tomu, že každá z dříve zmíněných otázek odpovídá jednomu pilíři observability, je jejich pořadí pro pochopení stavu systému zcela určující. Považuji za bezpředmětné ptát se" „proč se to děje", pokud nemáme jasnou odpověď na otázku: „co se děje". Stejně tak postrádá smysl lokalizovat chybu (ptát se „kde se to děje"), pokud přesně nevíme, jaký konkrétní děj v systému hledáme. Observabilita by proto měla být, dle mého názoru, vnímána jako diagnostický postup, který postupně přibližuje okolnosti incidentu.

3.2.1 Logy

Základním a nejčastějším výstupem, který slouží k diagnostice nasazeného řešení, jsou logy. Jedná se o textové či datové záznamy událostí, které mapují průchod programu jednotlivými stavy. Ačkoliv jsou tradiční logy často pouze nestrukturovaným textem, pro efektivní strojové zpracování a rychlou navigaci v datech je nezbytné využívat strukturované logování s dodatečnými metadaty. Mezi ně patří zpravidla časové razítko, úroveň závažnosti nebo názvy konkrétních entit a modulů. Strukturovaná podstata umožňuje vývojářům logy efektivně filtrovat a agregovat, což zkracuje čas potřebný k nalezení konkrétních událostí.

Aby logy věrně popisovaly dění v systému, vyžadují disciplinované využití. Zásadním cílem je záznam každého selhání. Pokud systém chyby tiše potlačuje, vytváří falešnou iluzi stability. Opačným extrémem je však nadměrné logování. V záplavě nepodstatných zpráv se pak obtížně hledají podstatné informace. S rozvojem systému navíc roste různorodost záznamů,

která odráží odlišné návyky vývojářů. Je proto nutné, aby logy byly pravdivé, jednotné a striktně se vyhýbaly úniku citlivých dat, jako jsou hesla či klíče k externím službám. (Gu et al., 2022)

3.2.2 Metriky

Metriky představují kvantitativní ukazatele, které měří vlastnosti běžícího systému a v diagnostické vertikále odpovídají na otázku, proč se události dějí v širším kontextu. Zatímco logy jsou popisem konkrétních událostí, metriky nabízejí numerický pohled na výkon, jako je počet požadavků za sekundu či vytížení systémových prostředků. Rozlišujeme metriky přímo měřené a metriky odvozené, které lze získat například agregací logů v čase. Klíčovou vlastností metrik je jejich schopnost úsporně uchovávat data ve formě časových řad. Díky tomu můžeme sledovat trendy, identifikovat anomálie a definovat prahové hodnoty pro automatizované varování (alerting), což je pro udržení stability produkčního prostředí zásadní.

Zaznamenávání metrik má smysl pouze u veličin, které se mění v čase běhu programu, ať už na základě konkrétních událostí, nebo v pravidelných intervalech. Z tohoto důvodu do observability neřadíme výsledky statické analýzy. Například složitost kódu měřená pomocí Halsteadových vzorců se během běhu programu nemění a už vůbec neposkytuje informaci o aktuálním stavu procesu. Naopak dynamické veličiny, jako je zaplnění operační paměti nebo latence síťových operací, jsou proměnlivé a jejich sledování odhaluje chování systému v delším časovém horizontu.

3.2.3 Trasy

Trasy jsou komplexním popisem události, který zachycuje kontext vzniku a související události. Trasa, narozdíl od logu, nabízí ucelený pohled na celou cestu a interakce v rámci jednoho děje. Vizualizuje tak kauzální vazby mezi komponentami, které by jinak zůstaly skryty, a umožňuje identifikovat přesná místa vzniku prodlev či selhání. Trasy zároveň slouží k vyčíslení času, který jednotlivé části systému vyžadují k vyřízení úlohy, a odhalují třecí plochy v komunikaci mezi moduly. (Madupati, 2023)

Specifickou variantou trasování je profilování, které se zaměřuje na vnitřní výkonnostní charakteristiky programu. Zatímco standardní trasy sledují cestu události mezi moduly, profily měří přesný čas provádění instrukcí, využití procesoru či alokaci paměti.

4 AUDIT ARCHITEKTURY SYSTÉMU

Tato kapitola se zabývá analýzou existujícího řešení. Cílem je stávající řešení zmapovat, modelovat pomocí diagramů, a popisovat ho na dostatečně detailních úrovní abstrakce, která mi dovolí poukázat na přítomný technický dluh. K zmapování a rozdělení na úrovně abstrakce jsem využil C4 diagramy, které navrhnul Simon Brown k popisu architektury na celkem čtyřech úrovních abstrakce (Brown, 2024c).

Ve světe softwarového inženýrství je již zavadený standard, který slouží k modelování softwarové architektury, a to je *Unified Modelling Language*. Ten slouží k modelování a dokumentaci jednotlivých částí a procesů. Modeluje tak jak vztahy mezi třídami, tak zároveň umožňuje zaznamenat komunikaci mezi komponentami pomocí sekvenčního diagramu či cestu uživatele pomocí user journey diagramu (Špánek, 2022). Proč jsem tedy nezvolil tento standard?

Jak popisuje Simon Brown v úvodu do C4 diagramů, týmy očekávají jednoduchost a srozumitelnost zápisu (Brown, 2024c). UML diagramy slouží zároveň jako dokumentace, což je jeho síla, a zároveň jeho slabina. Dokumentace musí být dostatečně podrobná a jednoznačná, s čímž přichází určitá dávka složitosti zápisu. Diagramy se stávají více deskriptivní a více nahuštěné, a zároveň se vytváří diagramy pro každý aspekt softwarového produktu. Množství diagramů tak rapidně stoupá a jejich udržování se stává časově i zdrojově náročnější. Pokud navíc dodržujeme netriviální notaci (například symboly panáčků nebo obdélníků s lištami a popisky), je o něco časově náročnější dokreslit nový blok nebo ho najít v knihovně komponent.

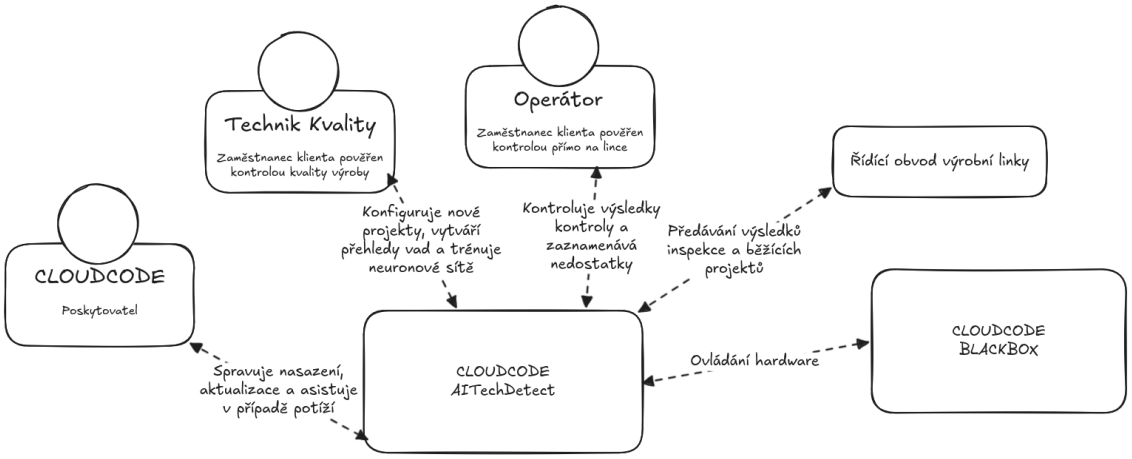
C4 byl navržen právě proto, aby se předešlo složité symbolice a velkému počtu diagramů. Každý diagram je de facto blokový diagram, který se skládá z jednoduchých tvarů a šipek. UML produkuje velké množství diagramů pokrývající každý aspekt zvlášť, zatímco C4 produkuje čtyři diagramy, které ale popisují řešení vždy v celku.

4.1 MODELOVÁNÍ SOFTWAROVÉ ARCHITEKTURY

C4 diagramy dělí softwarovou architekturu na vrstvu systémového kontextu, kontejnerů, komponent a kódu (Brown, 2024c). Diagram lze sestrojit pro každou úroveň zvlášť, přičemž diagram komponent a kódu se dají následně dělit podle jednotlivých kontejnerů a respektive komponent.

4.1.1 Úroveň systémového kontextu

Prvním krokem analýzy je zmapování systému v jeho přirozeném prostředí a přesné vymezení hranic mezi stávajícím řešením a okolními entitami. Tento systémový kontext slouží k vizualizaci obousměrných závislostí: od řešení směrem k externím službám i naopak. Takový pohled umožňuje posoudit kritičnost systému v širším technologickém ekosystému a identifikovat potenciální rizika plynoucí z vnějších vazeb (Brown, 2024d).



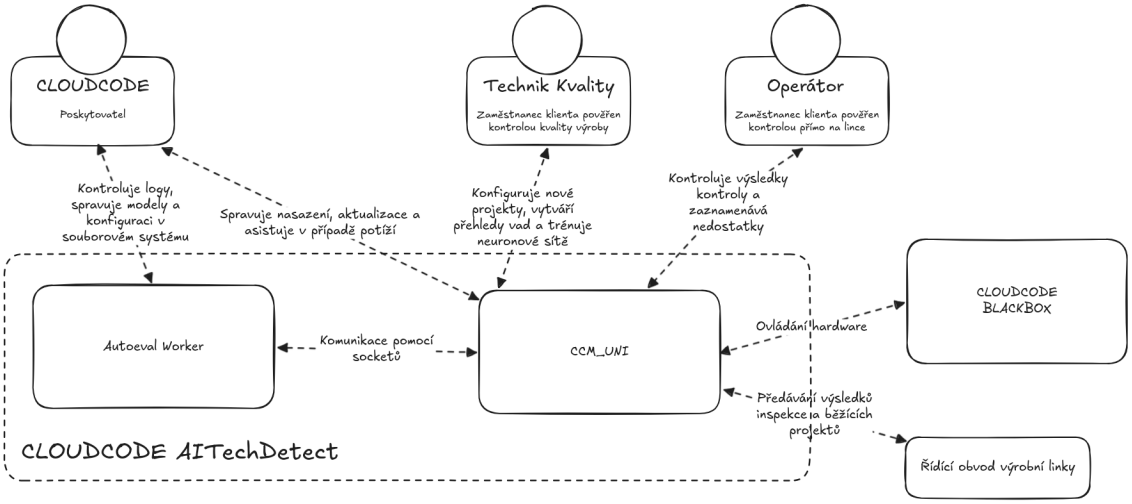
Obrázek 4.1: Diagram systémového kontextu

AITechDetect je koncipován jako izolovaná instalace přímo na hostitelský operační systém. Zatímco dřívější nasazení probíhala na kombinaci distribucí Linux (primárně Ubuntu LTS) a Windows 10, současným standardem je výhradně platforma Windows 11. Tento přechod byl motivován především obavami ze stability ovladačů grafických karet NVIDIA a obecně širší podpory na systémech Windows.

Už z podstaty řešení je jasné, že bude existovat v prostředí výrobní linky, a proto bude s největší pravděpodobností komunikovat s počítači PLC a dalšími nasazenými systémy, jako je například MES. Co se dotčených uživatelů týče, pohybuje se okruh lidí zpravidla ve čtyřech kategoriích: Operátor kvality, neboli dělník na výrobní lince; Mistr výroby, neboli nadřízený operátorům; Technik kvality, který zajišťuje politiku řízení kvality; a zaměstanci CLOUDCODE, kteří nasazené řešení vzdáleně spravují.

4.1.2 Úroveň kontejnerů

Pokud přiblížíme pohled na celkový systém, získáme úroveň kontejnerů. Cílem tohoto diagramu je ukázat, jaké jednotlivé části v systému spolupracují, jak spolu komunikují, a jaké mají odpovědnosti (Brown, 2024b).



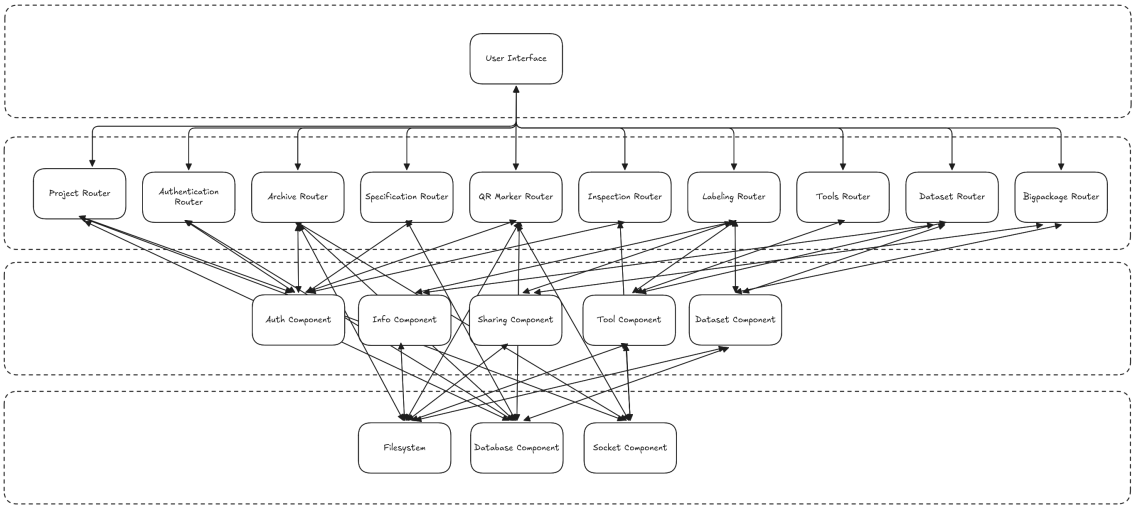
Obrázek 4.2: Diagram komponent

V případě AITechDetect nelze hovořit o úrovni kontejnerů, ale o úrovni procesů. Standardní distribuce zahrnuje dva běžící procesy: monolitické aplikační jádro *ccm-uni* a podpůrnou aplikaci *autoeval*. Zatímco jádro obstarává webové požadavky, práci s databází a souborové operace, *autoeval* slouží k práci s neuronovými sítěmi (Daněk, 2024). Komunikace mezi nimi probíhá prostřednictvím socketů. Již tato architektura odhaluje rozkol v přístupu k údržbě systému. Zaměstnanci zákazníka využívají primárně jádro, avšak pracovníci společnosti CLOUDCODE musí při správě často zasahovat přímo do procesu *autoeval*. Tyto manuální zásahy jsou typicky vynuceny neočekávanými pády či výjimkami, které nebyly zachyceny vnitřním testováním.

Absence systematického dohledu nad těmito stavy poukazuje na zásadní nedostatek: chybějící monitoring aplikačních procesů. Jak bylo uvedeno v kapitole 3.2, čím víc prvků observability máme k dispozici, tím více je pro nás systém transparentní při diagnostice. V současné architektuře si však observabilitu řeší každá služba zvlášť, což znemožňuje získat ucelený přehled o zdraví celého ekosystému. Tento deficit se projevuje i v samotném sběru diagnostických dat. Veškeré logy se zapisují lokálně do souborů, čímž dochází k devalvaci přínosů observability. Data zůstávají fragmentovaná v izolovaných úložištích, což značně komplikuje korelaci událostí a efektivní trasování příčin chyb v reálném čase.

4.1.3 Úroveň komponent

Při dalším detailním pohledu se dostáváme na úroveň komponent. Tento diagram rozkládá vybrané kontejnery na jejich vnitřní stavební kameny, čímž odhaluje konkrétní logické celky, jejich rozhraní a vzájemné vazby uvnitř jednoho běžícího procesu. Tím se nám vykreslí obrázek o vnitřním uspořádání aplikace. (Brown, 2024a)



Obrázek 4.3: Diagram komponent kontejneru CCM UNI

Ačkoliv se diagram může zdát kvůli obtížně interpretovatelný, poskytuje nezbytný nadhled nad architekturou aplikačního jádra. Na první pohled lze sice identifikovat rysy vrstvené architektury (viz kapitola 2.4.2), avšak odpovědnosti jednotlivých úrovní nejsou dodrženy. Detailní pohled na vazby mezi komponentami odhaluje zásadní strukturální deficit: v nejvyšší vrstvě určené pro směrování požadavků (routing) koexistují dvě nekonzistentní skupiny prvků. Zatímco jedna část respektuje hierarchii, druhá vrstvy zcela obchází a přistupuje přímo k nejnižší technologické úrovni. Důsledkem je totální eroze hranic; není zřejmé, kde jedna vrstva končí a druhá začíná.

Rozdělení do tří vrstev ovšem vyvolává zásadní otázku: kde se v celém modelu vlastně nachází business logika? Při analýze stávajícího třívrstvého uspořádání totiž zjišťujeme, že pro doménovou logiku zde fakticky chybí vyhrazený prostor. V důsledku této absence je nucena parazitovat na technologických vrstvách, čímž dochází k nechtěnému promíchání čistého návrhu s technickou implementací.

Celkově systém kvůli vysokému provázání působí dojmem, že nebyl budován podle jednotného plánu, ale spíše ad hoc podle aktuálních potřeb. Přítomnost těchto chaotických vazeb je jasným indikátorem úmyslného technického dluhu. Namísto čisté dekompozice byly voleny nejkratší cesty k cíli, což v architektuře vytvořilo nepřehlednou síť závislostí. Ta ve výsledku neslouží jako stabilní základ, nýbrž jako bariéra, která výrazně omezuje budoucí rozšiřitelnost a udržitelnost celého systému.

4.2 VYHODNOCENÍ RIZIK

Jednou věcí je systém popsat a vypíchnout jeho nedostatky. Úplně jinou disciplínou je však tyto problémy vzít a tento kategorický výčet rizik převést do porovnávatelné metriky. Právě k této ordinalizaci jsem se rozhodl využít metodu FMEA (Failure Mode and Effects Analysis), které se ve své kvalifikační práci věnoval můj dlouholetý kolega Matěj Vokál (2025).

FMEA je metoda původně vyvinutá pro vojenský průmysl, kterou si automobilový průmysl převzal a začal dále rozvíjet. V dnešní době je užívána druhá verze této metodiky, tzv. harmonizovaná FMEA. Ta analýzu možných selhání a rizik rozlišuje pro proces návrhu, logistiky, konstrukce či fungování systému (Vokál, 2025). Právě tato invariance vůči zkoumané oblasti nám dovoluje vzít principy metodiky FMEA a vytvořit defacto vlastní verzi pro zkoumání možných selhání a vad programového řešení.

Výstupem metody FMEA je matice, v níž každý řádek reprezentuje konkrétní systémovou položku a její potenciální způsob selhání. Pro každý bod je následně zpracována analýza vlivů, která postihuje jak lokální dopady v rámci komponenty, tak šíření poruchy do vyšších úrovní architektury. Kvantifikace rizik probíhá skrze definované parametry: závažnost dopadu, pravděpodobnost výskytu a obtížnost detekce. Z těchto faktorů je kalkulováno výsledné rizikové číslo, které slouží jako objektivní podklad pro prioritizaci implementace opravných opatření.

Standardní hodnotící škály metodiky FMEA zde ovšem nepostačí. Automobilový průmysl si je nastavil tak, aby zohlednil kvalitu provedení, fyzikální vlastnosti a vlivu na bezpečnost dílu a tím i celkového výsledku (Vokál, 2025). Tyto hodnotící škály jsem tudíž adaptoval pro potřeby zkoumání vad a příčin v rámci vývoje programového řešení a dopadu na vývojáře, nasazeného systému a koncového uživatele.

4.2.1 Závažnost vady

Závažnost vady popisuje dopad na stabilitu a provozuschopnost systému. Adaptovaná škála v tabulce 4.1 pokrývá několik milníků v životním cyklu programového řešení. Nejnižší stupně závažnosti (1–4) se zaměřují primárně na vnitřní kvalitu kódu a degradaci procesu údržby. V těchto případech vada sice neovlivňuje koncového uživatele, ale zvyšuje náročnost budoucího vývoje. Prostřední část stupnice závažnosti (5–8) modeluje situaci, kdy se projevy vady začnou šířit do uživatelské vrstvy: od zhoršení vedlejších funkcí až po úplnou ztrátu primární funkcionality systému. Vrchol škály závažnosti (9–10) je pak vyhrazen pro kritické stavy, kdy selhání přesahuje rámec aplikace a způsobuje externí škody. Jedná se tak zejména o porušení právních předpisů, nebo vznik bezpečnostního rizika pro společnost či uživatele.

Tabulka 4.1: Adaptovaná hodnotící škála závažnosti vady

S	**Definice**
1	**Bez následků**: Vada nemá žádný znatelný vliv na systém.
2	**Tolerovatelná údržba**: Změny v kódu jsou triviální.
3	**Nepříjemná údržba**: Změny v kódu jsou netriviální.
4	**Nepřijatelná údržba**: Změny v kódu jsou kaskádovitě netriviální.
5	**Zhoršení vedlejší funkce**: Částečná nefunkčnost neklíčové logiky.
6	**Ztráta vedlejší funkce**: Úplná nefunkčnost neklíčové logiky.
7	**Zhoršení primární funkce**: Částečná nefunkčnost klíčové logiky.
8	**Ztráta primární funkce**: Úplná nefunkčnost klíčové logiky.
9	**Právní riziko**: Porušení legislativních předpisů.
10	**Bezpečnostní riziko**: Ohrožení bezpečnosti jakékoliv strany.

4.2.2 Výskyt vady

Výskyt odhaduje pravděpodobnost projevu vady jako chyby v konečném produktu. V metodologii FMEA je tento parametr spjat s účinností preventivních opatření. Hodnota vyjadřuje frekvenci setkání s vadou při běhu programu; je-li například výskyt eliminován statickou analýzou, nabývá výskyt nízkých hodnot. Výsledek je tedy kombinací robustnosti prevence a četnosti aktivace daného kódu.

Stupně 1–2 označují vady blokované účinnými bariérami nebo kód spouštěný za výjimečných podmínek. Střední pásmo (3–6) reflektuje zřídka vznikající chyby, které preventivní mechanismy nezachytily. Nejvyšší hodnoty (7–10) jsou vyhrazeny pro kritické architektonické nedostatky, u nichž je selhání při běžném provozu prakticky nevyhnutelné. Tato gradace umožňuje exaktní prioritizaci nápravných opatření v závislosti na reálné expozici vady.

4.2.3 Detekce vady

Detekce hodnotí schopnost identifikovat vadu před jejím kritickým selháním v produkci. Škála reflektuje míru automatizace kontrol: nejnižší stupně (1–4) představují reaktivní mechanismy, kde chybu okamžitě zachytí lintery, kompilátor či monitoring. Střední pásmo (5–8) spoléhá na lidský faktor a poloautomatické nástroje, což s sebou nese riziko přehlédnutí. Nejvyšší hodnoty (9–10) označují skryté vady projevující se nekontrolovaně, pro jejichž odhalení neexistují relevantní metriky ani diagnostika.

Tabulka 4.2: Adaptovaná hodnotící škála výskytu vady

O	**Definice**
1	**Bez vlivu**: Prevence zcela eliminuje projev vady.
2	**Teoretický**: Výskyt pouze při souběhu vzácných podmínek.
3	**Sporadický**: Ojedinělá chyba v málo využívaném modulu.
4	**Občasný**: Náhodný projev při běžné interakci s aplikací.
5	**Střední**: Periodický výskyt závislý na běžném zatížení.
6	**Častý**: Pravidelný projev; absence preventivních mechanismů.
7	**Vysoký**: Opakované selhání klíčové funkce.
8	**Deterministický**: Vada se projeví při každém průchodu větví kódu.
9	**Kritický**: Téměř jisté selhání procesu ihned po inicializaci.
10	**Bariéra**: Vada znemožňuje start nebo základní exekuci systému.

Tabulka 4.3: Adaptovaná hodnotící škála detekce vady

D	**Definice**
1	**Okamžitá**: Vada znemožňuje kompilaci nebo exekuci kódu.
2	**Automatická (statická)**: Vada odhalena při vzniku.
3	**Automatická (start)**: Vada odhalena při inicializaci.
4	**Automatická (běh)**: Vada odhalena po výskytu.
5	**Poloautomatická**: Vada odhalena při cíleném testování.
6	**Manuální**: Vada je odhalena člověkem.
7	**Nespolehlivá (nástroj)**: Nástroje existují, ale detekce je nestálá.
8	**Nespolehlivá (člověk)**: Detekce závisí na náhodném nálezu při manuálním užití.
9	**Nahodilá**: Velmi malá pravděpodobnost zachycení.
10	**Nemožná**: Vada je v rámci stávající architektury nezjistitelná.

4.2.4 Zpracování analýzy

Adaptací hodnoticích škál FMEA jsou zkompletovány všechny nástroje nezbytné pro analýzu zdrojového kódu. Samotný proces analýzy připomíná princip kompilace: zdrojový kód je nejprve reprezentován diagramem objektů a morfismů, který plní obdobnou roli jako abstraktní syntaktický strom. Z této reprezentace se následně odvozují analýzy pro příslušnou úroveň abstrakce. Na úrovni kódu se hledá porušení pravidel clean code, zatímco na úrovni komponent se hledají architektonické protivzory vhodné k refaktorování. Konkrétní vady jsou pak přiřazeny jednotlivým nedostatkům a formalizovány formou záznamu do analýzy FMEA.

Serverový modul

Archivní modul

4.3 MODELOVÁNÍ DOMÉNOVÝCH PROCESŮ

5 ARCHITEKTURA NOVÉHO ŘEŠENÍ

5.1 POSTUPNÉ NASAZENÍ

6 ZÁVĚR

BIBLIOGRAFIE

BROWN, Simon, 2024a. *Component diagram* [online]. 2024-08-14. [cit. 2026-03-13]. Dostupné z: https://c4model.com/diagrams/component.

BROWN, Simon, 2024b. *Container diagram* [online]. 2024-08-14. [cit. 2026-03-13]. Dostupné z: https://c4model.com/diagrams/container.

BROWN, Simon, 2024c. *Introduction* [online]. 2024-08-14. [cit. 2026-03-13]. Dostupné z: https://c4model.com/introduction.

BROWN, Simon, 2024d. *System context diagram* [online]. 2024-08-14. [cit. 2026-03-13]. Dostupné z: https://c4model.com/diagrams/system-context.

COCKBURN, Alistair, 2005. *The Hexagonal Architecture* [online]. 2005-09-04. [cit. 2026-04-17]. Dostupné z: https://alistair.cockburn.us/hexagonal-architecture.

COPELAND, Lee, 2004. *A practitioner's guide to software test design*. Artech House.

DANĚK, Kevin, 2024. *Rapidní vývoj algoritmů strojového vidění* [online]. 2024-06-11. [cit. 2026-03-02]. Dostupné z: https://dspace.tul.cz/handle/15240/175790.

ELASTIC OBSERVABILITY TEAM, 2024. *The 3 pillars of observability: Unified logs, metrics, and traces* [online]. 2024-10-10. [cit. 2025-08-09]. Dostupné z: https://www.elastic.co/blog/3-pillars-of-observability.

FORRESTKNIGHT, 2025. *Everything You NEED to Know About TECH DEBT* [online]. 2025-07-24. [cit. 2025-08-10]. Dostupné z: https://www.youtube.com/watch?v=ukgmp6uxQJc.

FOWLER, Martin, 2009. *Technical Debt Quadrant* [online]. 2009-10-14. [cit. 2025-08-10]. Dostupné z: https://martinfowler.com/bliki/TechnicalDebtQuadrant.html.

GEEKSFORGEEKS, 2025. *Types of Software Architecture Patterns* [online]. 2025-04-07. [cit. 2026-04-10]. Dostupné z: https://www.geeksforgeeks.org/software-engineering/types-of-software-architecture-patterns/.

GU, Shenghui et al., 2022. Logging practices in software engineering: A systematic mapping study. *IEEE transactions on software engineering*. Roč. 49, č. 2, s. 902–923.

JAMES LEWIS, Martin Fowler, 2014. *Microservices: a definition of this new architectural term* [online]. 2014-03-25. [cit. 2026-04-20]. Dostupné z: https://martinfowler.com/articles/microservices.html.

JANSEN, Anton; BOSCH, Jan, 2005. Software architecture as a set of architectural design decisions. In: *5th Working IEEE/IFIP Conference on Software Architecture (WICSA'05)*. IEEE, s. 109–120.

KHAN, Sardar Mudassar, 2023. *Popular Software Architecture Used In Software Development*. ISBN 9798394479823.

MA, Noah, 2026. *Category* [online]. 2026-04-04. [cit. 2026-04-04]. Dostupné z: https://docs.racket-lang.org/ctp/_Category_.html.

MADUPATI, Bhanuprakash, 2023. Observability in Microservices Architectures: Leveraging Logging, Metrics, and Distributed Tracing in Large-Scale Systems. *European Journal of Advances in Engineering and Technology*. Roč. 10, č. 11, s. 24–31.

MÄNTYLÄ, Mika, 2010. Empirical software evolvability - Code smells and human evaluations. In: s. 1–6. Dostupné z DOI: 10.1109/ICSM.2010.5609545.

MARTIN, Robert C.; COPLIEN, James O., 2009. *Clean code: a handbook of agile software craftsmanship*. Prentice Hall. ISBN 9780132350884.

MICHIGAN TECHNOLOGICAL UNIVERSITY, 2025. *What is software engineering?* [online]. 2025-04-09. [cit. 2026-03-03]. Dostupné z: https://www.mtu.edu/cs/undergraduate/software/what/.

MIKKONEN, Tommi, 1998. Formalizing design patterns. In: *Proceedings of the 20th international conference on Software engineering*. IEEE, s. 115–124.

MYERS, Glenford J et al., 2004. *The art of software testing*. Sv. 2. Wiley Online Library.

NIERSTRASZ, Oscar, 1992. Towards an Object Calculus. In: ISBN 978-3-540-55613-8. Dostupné z DOI: 10.1007/3-540-55613-3_1.

OLUWATOSIN, Haroon Shakirat, 2014. Client-server model. *IOSR Journal of Computer Engineering*. Roč. 16, č. 1, s. 67–71.

PALLAPROLU, Sudhakar, 2025. Event-driven architecture: A modern paradigm for real-time responsive systems. *World Journal of Advanced Engineering Technology and Sciences*. Roč. 15, č. 2, s. 2860–2867.

PEREZ, Javier, 2011. *Refactoring Planning for Design Smell Correction in Object-Oriented Software*. Dis. pr.

SHVETS, Alexander, 2018a. *Code Smells* [online]. 2018-02-08. [cit. 2026-03-03]. Dostupné z: https://refactoring.guru/refactoring/smells.

SHVETS, Alexander, 2018b. *Code Smells* [online]. 2018-02-08. [cit. 2026-03-03]. Dostupné z: https://refactoring.guru/design-patterns/what-is-pattern.

SONARSOURCE, 2025. *Measuring and Identifying Code-level Technical Debt: A Practical Guide* [online]. 2025-10-21. [cit. 2026-03-25]. Dostupné z: https://www.sonarsource.com/resources/library/measuring-and-identifying-code-level-technical-debt-a-practical-guide/.

ŠPÁNEK, Roman, 2022. *Přednáška č. 5: UML I.* [Přednáška v rámci předmětu Softwarové inženýrství]. [cit. 2026-03-13]. Technická univerzita v Liberci.

TEAM ASANA, 2025. *What is technical debt? How to pay it off (with examples)* [online]. 2025-06-25. [cit. 2025-08-10]. Dostupné z: https://asana.com/resources/technical-debt.

TORVALDS, Linus, 2007. *Re: Coding Style* [online]. 2007-06-15. [cit. 2026-03-03]. Dostupné z: https://yarchive.net/comp/linux/coding_style.html.

VOKÁL, Matěj, 2025. *Zpracování harmonizované FMEA pro kryt a podlahu zavazadlového prostoru vozu Škoda Elroq dodávaného IMG AUTOMOTIVE s.r.o.* [online]. [cit. 2026-03-14]. Bakalářská práce. Univerzita Jana Evangelisty Purkyně. Fakulta strojního inženýrství. Ústí nad Labem.

A TEST

<table>
<tr><td colspan="2">ID:
ccm-00X</td><td>Scope:
Auth Module</td></tr>
<tr><td colspan="3">Popis vady (Failure Mode):
Chybějící sanitizace vstupu v přihlašovacím formuláři.</td></tr>
<tr><td colspan="3">Příčina vady (Failure Cause):
Nedostatečné využití bezpečnostních knihoven.</td></tr>
<tr><td colspan="3">Následek vady (Failure Effect):
Pád databáze při specifickém vstupu.</td></tr>
<tr><td>Lokální dopad:
Zablokování vlákna.</td><td>Dopad na vyšší úroveň:
Nefunkčnost autentizační služby.</td><td>Systémový dopad:
Kompletní nedostupnost systému.</td></tr>
<tr><td colspan="3">Současná preventivní a detekční opatření:
Manuální code review.</td></tr>
<tr><td>Závažnost (S):
8</td><td>Výskyt (O):
4</td><td>Detekce (D):
7</td></tr>
<tr><td colspan="3">Kritičnost (RiskPriorityNumber): 224</td></tr>
</table>